第 1 讲：整数与汇编入门

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

1. 先知道我们正在看什么
2. 二进制和十六进制：先会读位串
3. 位运算和逻辑运算：长得像，处理对象不同
4. 无符号与补码：理解负数为什么这样存
5. C 类型转换：先定类型，再算数
6. 运算、溢出与移位
7. 字节序：变的是存放顺序，不是数值
8. 汇编入门：先读宽度，再读源与目的
9. 独立练习：6 道短题 + 1 段追踪
10. 达标后的 2 道可选加练
11. 收束：做题时按这个顺序检查
12. 对照来源与回查位置

本讲对应复习第 1 天，覆盖课程第 2 讲及第 4 讲的基础，不是课程第 1 讲。先读懂“同一串位怎样被解释”，再读懂“一条指令究竟改变了什么”。这两个问题解决后，后面的条件、循环、调用和数组就有了落脚点。

本讲的文字、图和教学推导为助手整理；标为“助手补充”的例题与练习是教学自编。真题改编会单独注明出处。独立题答案在第 1 讲练习答案，请写完自己的尝试再打开。

1. 先知道我们正在看什么

1.1 程序、CPU、寄存器与内存

C 程序里的 `x = x + 3;` 描述一件事：取得原来的 x，加上 3，保存结果。CPU 不直接阅读这句 C，而是执行编译器生成的机器指令。汇编是机器指令的文字表示，适合人阅读。本讲不用你会写完整 C 程序，只需要逐渐认出变量、运算和指针。

CPU 是执行指令的部件。寄存器在 CPU 内部，数量少，每个能保存固定宽度的位；内存可以先看成一排有编号的小格子，每格保存 **1 字节 = 8 位**。地址是格子的编号，内容是格子里存的位。编号和内容可能长得一样，作用却不同。

图 D1–F1：读一个内存中的整数。



这张图的意思是：**用寄存器rdi中保存的地址找到内存，把从那里读到的4字节数据复制到寄存器eax中。** `%rdi`、`%eax` 是CPU内部存储位置的名字，前面的 `%` 是本套汇编语法的寄存器标记，不是求余。`movl` 中的 `mov` 表示复制，最后的小写字母L表示处理4字节（32位）；AT&T写法中左边是来源，右边是目的。括号 `(%rdi)` 表示“把rdi中的数当地址，到内存读取”，没有括号的 `%rdi` 才表示直接使用寄存器里的数。

本例rdi保存 `0x1000`，所以连续读 `0x1000`～`0x1003` 的四个格子。`0x` 表示十六进制；内容栏的 `05`、`00` 也按十六进制读。四个字节共同表示一个32位整数。**小端**指权重最低的字节放在最低地址：本例从低地址到高地址的字节为 `05 00 00 00`，其权重依次是1、256、256^2、256^3，所以数值为 $5\times1 + 0\times256 + 0\times256^2 + 0\times256^3 = 5$。按通常高位在左的写数方式，就是 `0x00000005`。这里的“还原”是按规则解释数据，不会重新排列内存。

执行后eax得到5，rdi仍是地址 `0x1000`，内存中的四个字节也不变，因为读取复制不会清空来源。字节序与寄存器的更多规则分别在第7、8节展开。

初学补充：Q004：图中的movl、寄存器与“小端还原”是什么意思？ 从“格子的编号”和“格子里的内容”开始，逐项拆解这张图。

解释补全（2026-10-07）：补充第一张图所需的汇编符号、读取宽度、小端读法与执行前后状态，原图数据和结论不变。

对应 C 可以写成 `int *p = ...; int x = *p;`。`int *p` 表示 p 是指向 int 的指针；`*p` 表示读取它指向的 int；`&x` 表示 x 的地址。声明中的 `*` 和表达式中的乘号不能混看。指针保存地址，但指针的类型还告诉编译器：一次读多少字节、指针加 1 跨过多少字节。

本套讲义统一采用课程的 **x86-64、Linux、AT&T 汇编语法**。常见基本类型宽度如下；这是本课程环境，不是所有 C 实现都必须如此。

类型	字节数	位数	今天先记住什么
`char`	1	8	普通 char 是否有符号依实现；需要明确时写 signed/unsigned char
`short`	2	16	整数运算前常先提升为 int
`int`、`unsigned int`	4	32	有无符号改变解释及部分运算规则
`long`、`unsigned long`	8	64	本课程的 Linux x86-64 采用此宽度
指针，例如 `int *`	8	64	指针大小与它指向的 int 大小不同
`float` / `double`	4 / 8	32 / 64	下一讲学习；不能按补码整数解释其数值

1.2 必须分开的三层问题

一道题有时问机器寄存器结果，有时问 C 表达式。这不是同一层。

1. **数学整数**没有本题规定的位数限制，例如 127 + 1 = 128。
2. **定宽机器位运算**只保留指定宽度。8 位加法保存低 8 位，结果的位串再按题目要求解释。
3. **C 语言表达式**先按类型、整数提升和语言规则求值。无符号运算按模回绕；有符号溢出不能普遍当作“自动回绕”，它是未定义行为。

“未定义行为”不是某个特殊数值，而是语言不再保证程序有什么结果。考试若明确要求“按 w 位补码机器运算”，就计算该机器位模式；若问标准 C 程序是否总成立，就不能拿机器回绕替 C 作保证。本讲凡采用补码位模型或负数算术右移，都明确说明。

2. 二进制和十六进制：先会读位串

2.1 每一位都有自己的权重

十进制的 305 是 `3×100 + 0×10 + 5`。二进制的 `101101` 是 `1×32 + 0×16 + 1×8 + 1×4 + 0×2 + 1 = 45`。最右边是第 0 位，权重为 1；往左依次是 2、4、8、16。

十六进制使用 `0–9`、`A–F`，其中 A=10，…，F=15。因为 $16=2^4$，**一位十六进制恰好对应四位二进制**。写 `0x2D` 时，`0x` 只是表示后面采用十六进制，不是数值的一部分。

十六进制	0	1	2	3	4	5	6	7
二进制	0000	0001	0010	0011	0100	0101	0110	0111
十六进制	8	9	A	B	C	D	E	F
二进制	1000	1001	1010	1011	1100	1101	1110	1111

`1011 0110` 按每四位一组就得到 `0xB6`。反过来，`0x03A7` 是 `0000 0011 1010 0111`。不要先转十进制再绕回去。补前导零不改变无符号数值，但在定宽题里能帮助你看清最高位。

常用数量：2^8=256，2^{10}=1024，2^{16}=65536，2^{32}=4294967296。遇到很大的数，不必硬写十进制，`2^32 − 24` 往往更清楚。

2.2 位串本身没有“天生的正负”

同一位串 `1110 1011`，按 8 位无符号整数解释是 235，按 8 位补码解释是 −21。改变的是解释方式，不一定是存储的位。后面遇到强制转换、联合、浮点位模式时，都要先问：“改的是数值类型，还是只换了读法？”

3. 位运算和逻辑运算：长得像，处理对象不同

`& | ^ ~` 分别表示逐位与、或、异或、取反。它们对每一位分别操作。与要求两位都是 1；或要求至少一位是 1；异或要求两位不同。逻辑运算 `&& || !` 先把整个操作数判断为“零或非零”，结果是 int 类型的 0 或 1。

用 8 位模式演示：

按列看：每次只处理对应的两个位

a	0	0	1	1	1	1	0	0	输入
b	0	0	0	0	1	1	1	1	输入
a & b	0	0	0	0	1	1	0	0	两边都是 1
a | b	0	0	1	1	1	1	1	1	至少一边是 1
a ^ b	0	0	1	1	0	0	1	1	两边不同
~a	1	1	0	0	0	0	1	1	只展示 8 位取反

位串从左到右是第 7 位到第 0 位。这里是 8 位模式示意；真实 C 的 ~a 还要先检查整数提升。

但 `a && b` 是 1，因为两者都非零；`!a` 是 0；`!!a` 是 1。`~a` 不等于 `!a`。真实 C 中若 a 是 `unsigned char`，`~a` 之前通常先把 a 提升为 32 位 int，因此结果并不只是 8 位；想只保留低 8 位，需要明确截断，如 `(unsigned char)~a`。

逻辑运算还有**短路**：`A && B` 若 A 为假，不计算 B；`A || B` 若 A 为真，不计算 B。因此：

```
int d = 0;
int ok = (d != 0) && (12 / d > 2);
```

左边为假，右边除法不执行，ok 为 0。改成按位 `&` 就没有这种保护，两边都会求值，整数除零会造成未定义行为。后面解释条件传送是否适用时，还会用到这个思想。

掩码（mask）是一串配合位运算、控制哪些位保留或改变的0和1；它本身也是普通整数，不是特殊类型。 例如，只看8位模式，`1010 1101 & 0000 1111 = 0000 1101`：与1得到原位，与0得到0，所以掩码 `0000 1111` 让低4位留下、高4位清零。

下面按本课程32位 `unsigned int` 的x解释；常量后缀 `u` 表示无符号，`~` 是逐位取反，`<< k` 是左移k位。位编号从最右边的第0位开始，`0 ≤ k < 32`。

表达式	掩码怎样构造	结果
`x & 0xFFu`	`0x000000FF`，低8位是1，其余是0	保留低8位，高24位清零
`x & ~0xFu`	对 `0x0000000F` 逐位取反，得到 `0xFFFFFFF0`	低4位清零，其余保留
`x | (1u << k)`	从1左移k位，只有第k位是1	第k位设为1，其余不变
`x ^ (1u << k)`	同上，只有第k位是1	第k位翻转（0变1、1变0），其余不变

掩码中1的含义取决于操作：与 `&` 表示保留，或 `|` 表示设为1，异或 `^` 表示翻转。这里 `^` 不是乘方。这些表达式只计算结果，要改变x本身还需赋值，例如 `x = x & 0xFFu;`；“保留低8位”也不意味着把结果类型变成8位，结果仍为32位无符号整数。

初学补充：Q005：掩码是什么，四种掩码表达式分别怎样工作？ 用逐位对齐的例子拆解保留、清零、置1与翻转。

解释补全（2026-10-07）：补充掩码定义、常量后缀与位编号，并明确“设为1”和“翻转”的区别；原四种表达式的结论不变。

4. 无符号与补码：理解负数为什么这样存

4.1 用最高位的负权重理解补码

图 D1–F2：8 位模式 `1101 1011` 的两种解释。

同一位模式的两种数值解释

8 位模式保持一致；只有最高位的权重由 +128 变成 −128。

位位置	7	6	5	4	3	2	1	0
位值	**1**	**1**	**0**	**1**	**1**	**0**	**1**	**1**
无符号权重	128	64	32	16	8	4	2	1
补码权重	**−128**	64	32	16	8	4	2	1

无符号解释 **219**　　8 位补码解释 **−37**

补码加权　`−128 + 64 + 16 + 8 + 2 + 1 = −37`

原始模式为 1101 1011：无符号为 219，补码为 −37。最高位为 1 时，补码值等于无符号值减 256。

> 对照：只把最高位由 1 改成 0，位串成为 01011011，两种解释都等于 91。

读图时只取位值为 1 的列并累加权重。无符号结果是 219；补码结果是 −128+64+16+8+2+1=−37。只有最高位权重不同，因此最高位为 1 时，**补码值 = 无符号值 − 2^w**；最高位为 0 时两种解释相同。

w 位表示	最小值	最大值
无符号	0	2^w−1
补码	−2^(w−1)	2^(w−1)−1

8 位补码范围为 −128 到 127，比正数多一个负数。因此“最小负数取相反数”无法用同宽补码表示。例如 32 位 int 的最小值取负在 C 中会溢出；不要背成“所有负数都能安全取反变正”。

D1–E1：把 −37 写成 8 位补码，再读回来

助手补充，完整例题。

1. 正数 37 的 8 位表示是 `0010 0101`，即 `0x25`。
2. 对 8 位取反，得到 `1101 1010`；再加 1，得到 `1101 1011`，即 `0xDB`。
3. 反查：0xDB=219，最高位为 1，所以补码值是 219−256=−37。

“取反加一”是在固定位宽里构造相反数的便捷方法，不是负号占一位、剩下七位写绝对值。`1000 0001` 在 8 位补码中不是 −1。解题时写清位宽，再使用这条方法。

初学补充：Q001：为什么补码不是“负号加绝对值”？ 用位权表解释 `1000 0001` 为什么是 −127、−1 为什么写作 `1111 1111`，以及“取反加一”和位宽的含义。

4.2 扩展与截断：先决定保留什么

把数放进更宽的容器时，通常希望保留原数值。无符号数高位补 0；补码数复制原最高位，这叫符号扩展。例如 8 位 −37 扩展到 16 位是 `1111 1111 1101 1011`，即 `0xFFDB`。若补 0 得到 `0x00DB`，数值就变成了正数 219。

截断到更窄的 w 位则是保留低 w 位，之后再按目标类型解释。**截断不保证原数值不变。** 例如 `0x12DB`（4827）截到 8 位，位模式是 `0xDB`：无符号解释为 219，补码解释为 −37。这个**位模型**很好计算；但 C 语言把超范围整数转成有符号窄类型时，在教材所依据的传统 C 规则下，结果由实现定义，或引发实现定义的信号，不能对所有实现一概保证低位补码解释。本课程机器题采用通常的 x86 补码截断模型，题面若指定其他规则则按题面。

初学补充：Q002：截断为什么会改变数值？C、实现定义和 x86 截断模型是什么意思？ 从丢掉高位的示意图开始，分别解释机器位操作与C语言转换规则。

更正与澄清（2026-10-07）：明确截断不保证数值不变；补全传统C规则允许引发实现定义信号的情况。原例数值不变，核对依据见Q002。

扩大位宽和改变有无符号若同时出现，应按实际转换顺序处理。例如一个值为 −37 的 signed char 转成 unsigned int，是把**数值 −37 转成 32 位 unsigned**，得到模 2^{32} 的结果；不是先偷偷转成 unsigned char 的 219。若代码真的写了 `(unsigned char)c`，那才先变成 219。

5. C 类型转换：先定类型，再算数

5.1 整数提升不是可忽略的小细节

比 int 小的整数类型参与多数算术、位运算、比较前，先做整数提升。本环境里 int 能容纳所有 unsigned char 和 unsigned short 的值，所以它们也提升为 int。`unsigned char a=250, b=10;` 中，`a+b` 先得到 int 类型的 260。

如果再把260赋给 `unsigned char`，就要转换成只能表示0～255的8位无符号整数。C对这种无符号整数转换规定按模256处理。**本例中，“模256”就是除以256取余数：260 = 1×256 + 4，所以转换后得到4。** 这里用256，是因为8位共有2^8=256种位模式。这与只保留低8位的结果相同：260写成 `1 | 0000 0100`，竖线左边的1代表256，丢掉后留下的 `0000 0100` 就是4。加法的结果仍是260，转换到窄类型后才得到4。

初学补充：Q003：“模”是什么意思，为什么这里要模256？ 从除法的余数开始，把取模、低8位和整数提升的先后顺序接起来。

解释补全（2026-10-07）：原文直接使用“模256”，遗漏了术语含义及与8位范围的联系；现已补充，原计算结果不变。

这解释了为什么不能看见“两个 8 位变量相加”就断言 C 在 8 位中做加法。汇编中的 **`addb` 是8位加法指令：`add` 表示加，`b` 表示byte，即1字节（8位）**。例如AT&T写法 `addb $10, %al` 表示把常数10加到8位寄存器al中，结果写回al；若al原位模式按无符号解释为250，则只保存和的低8位，得到4。它直接指定8位机器加法，与C中先提升成int计算是不同层次。

初学补充：Q006：addb是什么，和C里的加法有什么区别？ 拆解指令后缀、操作数和250＋10的位级结果。

5.2 混合比较先做公共类型转换

在 32 位 int 与 unsigned int 的常见组合中，int 会转为 unsigned int。负数因此变成一个大的非负数。例如 `-2 < 1U` 比较时，左边按 32 位 unsigned 解释，结果为假。

不要扩展成“只要有 unsigned，整个表达式一定无符号”。本环境的 64 位 long 能容纳全部 32 位 unsigned int，所以 `-2L < 1U` 会先把右边转换为 long，结果为真。够用的判断顺序是：

1. 先提升 char、short 等窄类型。
2. 类型一样，直接按该类型计算。
3. 都有符号或都无符号时，转到等级更高的类型。
4. 一有符号一无符号：若无符号类型等级不低，转到无符号类型；否则检查**等级更高的有符号类型**能否表示无符号类型的全部值。能，则统一到这个有符号类型；不能，则双方统一到**这个有符号类型对应的无符号类型**。

等级与位宽不是同一概念：常见等级为 `int < long < long long`，每种有符号类型与其对应无符号类型等级相同。本环境中long与long long都是64位，long long的等级仍更高。判断能否容纳时看**整个类型范围**，不是当前变量的值。例如long能容纳32位unsigned int，二者统一为long；long long不能容纳64位unsigned long的全部值，二者统一为unsigned long long。

初学补充：Q007：无符号类型等级较低时，究竟转成什么？ 展开“能装下”和“装不下”两条分支，区分等级、位宽与取值范围。

今天优先练熟 `int/unsigned int` 和 `long/unsigned int`。**整数常量只有U或u后缀时，依次从unsigned int、unsigned long、unsigned long long中选第一个能表示其值的类型**，候选中没有char或short。本环境中 `40U` 是unsigned int，`4294967296U` 则是unsigned long。没有后缀的小十进制常量通常是int；其他后缀或进制要按各自候选表判断。声明中的 `unsigned` 单独写是 `unsigned int` 的简写，与“U后缀常量可能选择不同类型”是两件事。

初学补充：Q008：整数常量只有u后缀，怎样判断具体类型？ 对照边界例子，以及常量类型、变量类型和unsigned简写。

更正与澄清（2026-10-07）：将“较宽的有符号类型”改为“等级更高的有符号类型”，避免混淆等级与位宽；补全仅U后缀的候选顺序。规则依据见Q007、Q008。

D1–E2：负数变 unsigned 后，为什么加法和比较看起来反常?

改编自 2025 期末无答案版（来源资料未随仓库分发），第一题第 1 小题，PDF 第 2 页。将原数值改为 −24 和 40，并把选择题改成逐步推导；不沿用原题评分。

```
int x = -24;
unsigned u = (unsigned)x;
unsigned v = u + 40U;
```

读代码提示：这里 `unsigned` 就是 `unsigned int`，变量名u只是名字；`40U` 因为40能被unsigned int表示，也是unsigned int。Q007：混合类型转换分支 · Q008：U后缀常量的具体类型。

求 u 的位模式、v，以及 `x < u`、`x == u` 的结果。

第一步：明确宽度。 int 和 unsigned 均为 32 位。负数转 unsigned 的值是对 2^{32} 取模，所以 $u=2^{32}-24$，十六进制为 `0xFFFFFFE8`。

第二步：在 unsigned 中加。 $u+40=(2^{32}-24)+40=2^{32}+16$。无符号结果保留模 2^{32} 的值，所以 v=16。不是算术坏了，而是有限范围的无符号加法就这样定义。

这里$2^{32}+16$是推导中的**数学和**，不是一个32位int已经保存的值；操作数u与40U都是unsigned int，因此本步是无符号运算。若换成C有符号int运算，必须检查实际运算类型的溢出，不能沿用本步保证。参见 Q009：数学结果达到$2^{32}+16$时如何处理 中的追问2。

第三步：比较前统一类型。 `x < u` 中 x 被转成 unsigned，转换后的 x 恰好等于 u，因此小于为假，等于为真。不能一边坚持 x 是负数，一边拿 u 的无符号值直接比较。

一个稳定的作答格式是：**原类型 → 转换后的类型与数值 → 运算 → 结果类型**。只写最终“真/假”，即使猜对也不容易发现思路哪里不稳。

初学补充：Q009：有符号结果超出范围时，也能直接取模吗？ 先核对无符号转换的理解，再区分补码机器运算、C有符号算术与超范围类型转换。w位补码的范围是−2^(w−1)～2^(w−1)−1，不是以2^w作为正负边界。

6. 运算、溢出与移位

6.1 位宽限制如何改变结果

无符号 w 位加法、乘法都可用“数学结果对 2^w 取模”计算。以 8 位位模型为例，240+32=272，保留低 8 位得到 16；20×20=400，保留低 8 位得到 144。

相同位加法结果换成补码解释，可能发生有符号溢出：120+20 的数学结果为 140，8 位结果为 `1000 1100`，补码解释是 −116。这里说的是 **8 位机器加法**。若写成 C 的 `int a=120; a+20`，32 位 int 完全装得下，结果就是 140；即使 a 是 char，整数提升也常使加法在 int 中进行。

判断补码加法溢出：两个正数相加得到负数，或两个负数相加得到非负数。异号相加不会产生有符号加法溢出。无符号进位和有符号溢出是两个问题：同一个位加法可以只有其中一种，也可以两种都有。下一讲用 CF、OF 给它们命名。

若题目明确要求**两个w位补码数的机器加法**，数学和超过最大值2^(w−1)−1时，正溢出的机器结果按补码读为“数学和−2^w”；例如8位的140变为140−256=−116。若问C中实际w位有符号加法，这种越界是未定义行为，不能保证回绕结果；先检查整数提升，别把变量原位宽误当成运算位宽。

针对“负方向超范围”和“C能否直接按机器回绕”的解释，见 Q009：有符号结果超出范围时，也能直接取模吗？ 。其中追问2、3分别计算数学结果$2^{32}+16$与$-2^{32}-6$只保留低32位后如何按补码读取。

6.2 左移、逻辑右移、算术右移

先用8位机器模式 `1101 1011` 观察移两位。**下面三行分别从同一个初态出发，不是连续执行，也没有给这串位统一指定C类型。** 原模式按8位无符号数读是219，按8位补码读是−37。

操作	得到的8位模式	空出来的位怎么填	按无符号读	按补码读
左移2位	`0110 1100`	右侧补0，超出左边的位丢弃	108	108
逻辑右移2位	`0011 0110`	左侧补0	54	54
算术右移2位	`1111 0110`	左侧复制原最高位；本例为1	246	−10

有符号整数类型允许表示负数、零和正数；**补码**是这些整数的一种二进制表示方法。本课程x86-64的有符号整数采用补码，因此“按补码读”就是按相应位宽的有符号整数读。“有符号”不等于当前值为负，“补码”也能表示正数和零。逻辑右移通常配合无符号解释，算术右移通常配合补码有符号解释；具体机器指令按指定规则移动位，不会自动知道C类型。

初学补充：Q010：移位表中是有符号还是无符号？补码与有符号有什么区别？ 对照原位模式的两种读法及三种移位结果。

解释补全（2026-10-07）：明确三行独立、区分位操作与数值解释，并补上两种读法的十进制值；原移位结果不变。

机器的左移可以用乘 2^k、保留低位理解。无符号逻辑右移相当于除以 2^k 再向下取整。补码算术右移也相当于向下取整，而 C 有符号整数除法从 C99 起采用**向零截断**，负数时二者可能不同。

D1–E3：为什么 −19 除以 8，不能直接照搬右移?

助手补充，按 32 位补码且负数算术右移的课程机器假设。

1. 精确商为 −2.375。向零截断是 −2，所以 C 的 `-19 / 8` 为 −2。
2. 算术右移 3 位相当于向下取整，得到 −3。它向更负的方向走了一步。
3. 对负数先加 2^3−1=7：−19+7=−12；再算术右移 3 位，得到 −2。

这多加的 7 称为偏置 bias：把负数所在的区间挪到能够向零截断的位置。一般除以 2^k 时，非负数直接右移；负数先加 2^k−1，再算术右移。

```
/* 仅采用本课程负数算术右移假设；这里限制 0 <= k <= 30。 */
int div_power2(int x, int k) {
    int bias = (1 << k) - 1;
    return (x + (x < 0 ? bias : 0)) >> k;
}
```

`条件 ? A : B` 表示条件为真取 A，否则取 B，只求值被选的一支。限制 k 是为了避免构造 bias 时让 `1 << k` 本身越过有符号可表示范围；负 x 加上这范围内的非负 bias 不会发生有符号加法溢出。

这里右移的左操作数 `x + (x < 0 ? bias : 0)` 是有符号int。按代码注释采用的课程实现假设，**若这个和为负，右移自动在左侧补1；非负则补0，不需要手动补符号位**。复制的是相加后实际被右移值的最高位。bias的作用是修正“向下取整”与“向零截断”的差别，不是实现补位；例如−19先加7得到−12，再算术右移3位得到−2。

初学补充：Q011：C代码中的右移会自动补符号位吗？bias是用来补位的吗？ 区分自动补位、左操作数类型与除法取整修正；传统C对负数右移的实现前提见6.3。

6.3 哪些移位规则不能混用

情境	应怎样作答
C 无符号左移	合法移位量内，按乘 2^k 后模 2^w；w 取提升后左操作数宽度
C 无符号右移	左补 0，相当于向下取整除以 2^k
C 有符号负数右移	教材所用传统 C 规则中是实现定义；本课程 x86 通常采用算术右移
C 对负有符号数左移，或非负有符号左移结果不可表示	按教材所用传统 C 规则，不作可靠回绕结果，属于未定义行为
C 移位量为负，或不小于提升后左操作数宽度	未定义行为；`1U << 32` 不能说等于 1 或 0
x86 指令 `shl/sal`、`shr`、`sar`	直接按机器指令追踪；sal/shl 左移，shr 逻辑右移，sar 算术右移

x86 的移位指令对移位计数有硬件处理规则，例如 32 位操作取计数低 5 位，64 位操作取低 6 位。**这不使越界移位的 C 表达式变得有定义。** 题目是 C 还是已经给定汇编，要先看清。

7. 字节序：变的是存放顺序，不是数值

图 D1–F3：把 32 位数 `0x1234ABCD` 放在地址 0x2000。



读图从地址最小的一格开始。小端把**最低有效字节**放在**最低地址**，x86–64 使用小端。大端把最高有效字节放在最低地址。两种表示存的整数都还是 `0x1234ABCD`，不会因为小端就变成 `0xCDAB3412`；后者只是把按地址列出的字节串误当成了通常书写的数。

拆字节时先从右往左每两位十六进制一组，再按地址放好。不要把每个字节内部的位再反转，也不要把每个十六进制字符独立反转。

考题写“第一个字节”时最好明确写“最低地址处字节”；写“最高字节”则明确是最高有效字节。符号扩展和字节序也要分步：先计算完整新位模式，再把它分成字节按小端排列。

8. 汇编入门：先读宽度，再读源与目的

8.1 寄存器名称中的宽度

寄存器不自带 C 类型。`%rax` 是 64 位寄存器；`%eax`、`%ax`、`%al` 是它的一部分，不是四个互不相干的盒子。

图 D1–F4：同一个 rax 的不同访问范围。



读图时以右端第 0 位对齐：eax 是低 32 位；ax 是低 16 位；al 是低 8 位；ah 是第 8—15 位。ah 只需认得，现代 64 位代码常用低字节寄存器，不必把它当主要工作对象。

64 位	低 32 位	低 16 位	低 8 位
rax	eax	ax	al
rbx	ebx	bx	bl
rcx	ecx	cx	cl
rdx	edx	dx	dl
rsi / rdi	esi / edi	si / di	sil / dil
rsp / rbp	esp / ebp	sp / bp	spl / bpl
r8—r15	r8d—r15d	r8w—r15w	r8b—r15b

一个高频规则：**写 32 位通用寄存器会把对应 64 位寄存器的高 32 位清零**。写 8 位或 16 位子寄存器通常保留其余位。例如原 rax=`0xAABBCCDD11223344`：

- `movb $0x77, %al` 后为 `0xAABBCCDD11223377`。
- 从原状态执行 `movw $0x7788, %ax` 后为 `0xAABBCCDD11227788`。
- 从原状态执行 `movl $0x778899AA, %eax` 后为 `0x00000000778899AA`。

以上三行彼此独立，不是连续执行。读 eax 不会清高位，**写 eax 才会**。

“读eax”是取出它的现有内容来用，“写eax”是把结果存进它。例如 `movl %eax, %ecx` 读eax、写ecx，不改变rax，但会清零rcx的高32位；`movl $5, %eax` 写eax，使完整rax变成5。即使 `movl %eax, %eax` 把原值写回，仍然发生了写eax，所以也会清零rax高32位。判断依据是有没有写入，不是低32位的数值是否改变。

初学补充：Q012：读eax与写eax有什么区别，为什么只有写才清高位？ 用同一初态分别追踪读取、写入和原值写回。

8.2 读 AT&T 的一条指令

`movq %rdi, %rax` 的方向是 **源在左，目的在右**，把 rdi 的 64 位内容复制到 rax。mov 不会把源清空，也不是交换。后缀 b/w/l/q 对应 1/2/4/8 字节；这里 l 表示 32 位，与 C 的 long 在本环境为 64 位并不矛盾。

操作数	示例	含义
立即数	`$8`、`$0x2000`	直接使用这个常数
寄存器	`%rax`	读取或修改寄存器的位
内存	`(%rdi)`、`8(%rdi)`	先算地址，再按指令宽度访问内存

`movq $0x2000, %rax` 把数 0x2000 放进 rax；`movq (%rdi), %rax` 才去内存取数。普通 mov 的源和目的不能同时是内存，要借一个寄存器中转。立即数也不能成为目的。

注意：`%rdi` 表示寄存器操作数，里面的值可以是地址，也可以是普通整数；不能直接把它定义成“地址”。在这里的mov指令中，`(%rdi)` 表示以rdi的值为地址的内存操作数，在源位置读内存，在目的位置写内存。初学补充：Q013：(%rdi)和%rdi的区别，后者一定是地址吗？ 。

两操作数的算术指令同样读成 `目的 = 目的 运算 源`：`subq %rcx,%rax` 是 rax−rcx；`addq` 相加，`imulq` 的双操作数形式留下乘积的低位，`andq/orq/xorq` 做位运算。这些指令还会影响条件码，今天先追踪数据，下一讲学习条件码。

8.3 复杂寻址只是一个加法公式

`D(Rb,Ri,S)` 表示地址 `D + Rb + Ri×S`。D 是字节位移；Rb 是基址寄存器内容；Ri 是索引寄存器内容；S 是比例因子，允许 1、2、4、8。省略基址或索引时对应项为零，省略比例通常按 1。

比例因子不是由 mov 的后缀自动决定的。 `movq 8(%rdi,%rcx,4),%rax` 仍然先算 `rdi+4×rcx+8`，然后从那里读 8 字节。

D1–E4：相同的括号，mov 和 lea 做的事为什么不同?

助手补充。 已知 rbx=0x3000、rcx=3，地址 0x3020 开始的 8 字节按小端表示整数 9。分别从这个初始状态执行：

```
movq 8(%rbx,%rcx,8), %rax
leaq 8(%rbx,%rcx,8), %rax
```

第一步都计算 `0x3000 + 3×8 + 8 = 0x3020`。随后 mov 去地址 0x3020 读取 8 字节，rax 得到 **9**；lea 直接把计算得到的地址值 **0x3020** 写入 rax，不读这块内存。

lea 不要求算出的数一定是有效内存地址，所以还能用于普通算术：`leaq (%rdi,%rdi,4),%rax` 得到 5×rdi 的低 64 位。括号在 lea 中给出的是计算公式，不是访存命令。lea 也不修改条件码，后面追踪比较时尤其要记住。

8.4 扩展指令：是补零还是补符号位?

movz 系列零扩展，movs 系列符号扩展。名字中最后两个字母通常给源宽度和目标宽度：`movzbl` 为 byte→long，即 8→32 位；`movsbq` 为 8→64 位；`movslq` 为 32→64 位。

假设 al 的位模式是 `0x90`：

指令，分别执行	执行后 rax	为什么
`movzbl %al,%eax`	`0x0000000000000090`	先 8→32 零扩展，再由写 eax 清高 32 位

指令，分别执行	执行后 rax	为什么
`movsbl %al,%eax`	`0x00000000FFFFFF90`	先 8→32 符号扩展，再清高 32 位
`movsbq %al,%rax`	`0xFFFFFFFFFFFFFF90`	直接 8→64 符号扩展

第二行最容易错：它确实做了符号扩展，但**目标只有 32 位**；随后写 eax 的规则仍然有效。32→64 零扩展常直接用 `movl` 写 32 位目标实现，不需要寻找不存在的普通 `movzlq` 指令。

D1–E5：逐行追踪必须保留中间状态

助手补充。 rdi=0x4000，地址 0x4000 的 4 字节表示 `0xFFFFFFF9`，初始 rax 的值不重要。

```
movl   (%rdi), %eax
movslq %eax, %rax
addq   $4, %rax
```

行	源是什么	执行后 rax	改了哪里
1	地址 0x4000 的 4 字节	`0x00000000FFFFFFF9`	写 eax，连带清 rax 高位；内存不变
2	eax 的 32 位补码值 −7	`0xFFFFFFFFFFFFFFF9`	符号扩展到整个 rax
3	立即数 4	`0xFFFFFFFFFFFFFFFD`，补码值 −3	修改 rax；内存不变

第一行之后，完整 rax 按 64 位补码解释仍是正数；第二行之后才是 64 位 −7。不要把“低 32 位可解释为 −7”与“整个寄存器就是 −7”混成一句话。

9. 独立练习：6 道短题 + 1 段追踪

以下 D1–P1—P7 为必做，共 **25 分，20 分为 80% 过关线**。P1—P4 是整数短题，P5—P6 是汇编短题，P7 是短追踪。每题写出关键过程；得到提示后完成的部分记为练习，不计入首次独立成绩。题目默认沿用本讲环境，所有内存地址均假定合法。

D1–P1：同一模式、两种解释与字节序（4 分）

助手补充。

1. 把 8 位 `0xB6` 写成二进制。（1 分）
2. 分别按 8 位无符号数、8 位补码解释，给出十进制值。（各 1 分）
3. 32 位数 `0x81A2C3D4` 存在地址 0x5000，请画出小端环境中 0x5000—0x5003 的四个字节。（1 分）

D1–P2：位运算、逻辑运算与短路（3 分）

助手补充，a、b 均为 unsigned int，a=0x28，b=0x14。

1. 写出 `a & b` 与 `a && b` 的值。（合计 1 分）
2. `int z=0; int r=(z!=0)&&(18/z>1);` 是否执行除法？r 为多少？（1 分）
3. 用一个按位与表达式将 unsigned int x 的低 4 位清零，其余位不变。（1 分）

本题追问：Q005：0x0u的写法合法吗，能用于这道题吗？（含第3小题解答，按需打开）。

D1–P3：提升、扩展和比较（3 分）

助手补充。

1. `signed char c=-18; int a=c;` 写出 a 的 32 位十六进制表示。(1 分)
2. `unsigned char p=240, q=30; int t=p+q;` 写出 t 的值，并说明在哪种类型中相加。(1 分)
3. 判断 `-3 < 2U` 和 `-3L < 2U`，各说明比较时采用的公共类型。(合计 1 分)

D1–P4：移位与运算层次（4 分）

助手补充。

1. 对 8 位机器模式 `1010 1100` 分别逻辑右移 2 位、算术右移 2 位，写出结果位模式。(合计 1 分)
2. 计算 C 的 `-25 / 8` 和本课程机器假设下的算术右移 `-25 >> 3`。若用 bias 修正后者，应先加多少，修正结果是多少？ (2 分)
3. 32 位环境下，C 的 `1U << 32` 能否保证结果为 1？写出依据。(1 分)

D1–P5：子寄存器写入（3 分）

助手补充。以下三问**分别**从 rax=`0x89ABCDEFFEDCBA98` 开始，写出执行后的完整 rax。

1. `movb $0x11, %al`。(1 分)
2. `movl $0x76543210, %eax`。(1 分)
3. `movsbl %al, %eax`。(1 分)

D1–P6：地址还是内容（3 分）

助手补充。rbx=0x200，rcx=3；地址 0x228 开始的 8 字节表示整数 0x8899。以下分别执行，写出 rax 的值，并注明是否读取数据内存。

1. `leaq 0x10(%rbx,%rcx,8), %rax`。(1 分)
2. `movq 0x10(%rbx,%rcx,8), %rax`。(1 分)
3. `movq $0x228, %rax`。(1 分)

D1–P7：5 行追踪（5 分）

助手补充。初始 rdi=0x1000，rsi=2；地址 0x1008 的 4 字节表示 `0x00000007`，地址 0x100C 的 4 字节表示 `0xFFFFFFFE`。按顺序执行：

```
leaq   (%rdi,%rsi,4), %rdx
movl   (%rdx), %eax
addl   4(%rdx), %eax
movl   %eax, (%rdx)
movslq 4(%rdx), %rcx
```

每行 1 分：列出源值/有效地址、执行后改变的寄存器或内存及其值。凡写 eax，请报告完整 rax；最后一行报告完整 rcx。必须把第 4 行的写内存与前几行的读内存区分开。

10. 达标后的 2 道可选加练

以下不计入 25 分。如果未达标，先修复 P1—P7 的错因，再用它们做针对性练习。

D1–A1：边界与“看起来相同的代码”

助手补充。32 位 int 最小值记为 INT_MIN，最大值记为 INT_MAX。

1. C 表达式 `-INT_MIN` 是否能保证得到正数？说明原因。
2. `unsigned int a=0xFFFFFFFFU;` 时 `a+1U` 的值是多少？为什么与 `INT_MAX+1` 的语言保证不同？

3. `unsigned char a=255;` 时 `a+1` 的类型和值分别是什么？如果赋给 unsigned char b，b 又是多少？

D1–A2：清低位等价于向下对齐

改编自 2025 期末无答案版（来源资料未随仓库分发），第一题第 3 小题，PDF 第 2 页。把移位量 5 改为 3，并改为解释题。

对 32 位 unsigned int x，令 `y=(x>>3)<<3`。

1. 写出与 y 等价的位掩码表达式。
2. 说明为什么 `y<=x`，以及 `x-y` 的范围。
3. 用 x=77 验证上述结论；再判断 x 为 8 的倍数时是否必有 y=x。

11. 收束：做题时按这个顺序检查

整数题先标 **类型和宽度**，再写位模式或数值；转换题写清先提升、再统一类型、最后赋值；溢出题确认问的是机器位结果还是 C 保证。字节序题先求完整模式，再拆成字节按地址摆放。

汇编题每行问四件事：**操作宽度是多少？源在哪里？有效地址是多少？目的的哪些位或哪些字节改变？** lea 只计算，mov 才按操作数决定是否访问内存；写 eax 记得清高位。

本讲通过的实际标准是：P1—P7 达到 20/25，能用自己的话说明有符号/无符号混合比较，能画对一次小端字节图，P7 能区分读内存、写内存、只改寄存器。若分数达标但仍分不清地址与内容，先修这一点再进入条件与分支。只记录真实出现的错因，不预先填“我已掌握”。

12. 对照来源与回查位置

以下页码全部是 **PDF 文件页序**，不是教材印刷页码。阅读本讲不要求先通读原书；只有概念需要回查时才打开对应小节。

本讲内容	已核对的课件位置	教材对应小节
进制、数据宽度	第 2 讲 PDF 第 3—5 页	2.1.1—2.1.2
位运算、逻辑运算、移位	第 2 讲 PDF 第 7—12 页	2.1.6—2.1.9
补码、范围、转换、扩展截断	第 2 讲 PDF 第 14—34 页	2.2.1—2.2.7
加乘溢出、移位乘除	第 2 讲 PDF 第 36—48 页；bias 推导为教材补充	2.3.1—2.3.7，特别是 2.3.7
内存、地址、字节序	第 2 讲 PDF 第 55—64 页	2.1.3；硬件背景参见 1.4.1
C 到机器指令、数据宽度	第 4 讲 PDF 第 12—21 页	3.2—3.3
寄存器、mov、寻址	第 4 讲 PDF 第 26—40 页	3.4.1—3.4.3；扩展细节见 3.4.2
lea、算术、移位指令	第 4 讲 PDF 第 42—46 页	3.5.1—3.5.3

原始来源：第 2 讲课件（来源资料未随仓库分发）、第 4 讲课件（来源资料未随仓库分发）、教材原书第 3 版中文版（来源资料未随仓库分发）。教材引用到小节，不猜填印刷页码。

本讲未使用 2025 第一次小测的具体题面或答案。后续针对学习中真实问题的补讲另存文件，本讲保留为最初教学版本。

第 2 讲：浮点与条件分支

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

1. 二进制小数：小数点右边也有位权
2. 存下来的 exp、frac，不等于公式里的 E、M
3. 解码和编码都用固定步骤
4. 范围和精度：浮点数是一把间距会变的尺
5. 舍入：先找相邻刻度，再判断中点
6. 浮点加乘与 C 转换：每次运算都可能丢信息
7. 零、无穷、NaN 与“看位”
8. 条件码：CPU 记下的四个一位结果
9. 同一次 cmp，为什么有符号和无符号会走不同分支?
10. setcc 写一字节，返回整数还需整理高位
11. if 与 cmov：先说明执行了什么，再谈结果
12. 独立练习：先交过程，再核对答案
13. 过关检查与资料定位

本讲是复习计划的第二天，对应课程第 3 讲和第 5 讲的前半部分，并非课程第 2 讲。

今天要完成两件事：把一个浮点位串可靠地还原成数值，并且看懂一段汇编为什么走某条分支。前一件事训练你区分“存下来的字段”和“字段表示的数”；后一件事训练你区分“计算得到的位”与“如何解释这些位”。它们都依赖第一天的补码、无符号数、寄存器宽度和操作数顺序。

先用自己的话回答三个入口问题：`0xff` 按 8 位无符号数和补码各表示什么？`movq (%rdi), %rax` 取的是地址还是内容？向 `%eax` 写入会怎样影响 `%rax` 的高 32 位？若仍不确定，先回到第 1 讲修这几个点。本讲不要求你已经会函数调用栈。

阅读时先学到一个例题并自己重算，再继续下一段。文末必做题共 30 分，首次独立达到 24 分为当天检查门槛；答案另放在第 2 讲练习答案。不要边做边看答案。这里没有使用 2025 第一次小测的具体题目。

1. 二进制小数：小数点右边也有位权

十进制的 `0.375` 是三项位权的和；二进制也一样，只是小数点右边的位权依次是 $2^{-1}, 2^{-2}, 2^{-3}, \ldots$。因此

$$101.011_2 = 1 \times 2^2 + 1 \times 2^0 + 1 \times 2^{-2} + 1 \times 2^{-3} = 5.375。$$

整数部分可以用除以 2 取余数法，小数部分可以用乘以 2 取整数部分法。例如 $0.375 \times 2 = 0.75$，先取 0；$0.75 \times 2 = 1.5$，取 1 并留下小数部分 0.5；$0.5 \times 2 = 1$，再取 1。因此 $0.375 = 0.011_2$。

有限长度的二进制小数只能精确表示分母约分后是 2 的幂的有理数。例如 $3/8$ 可以有限表示，$1/10$ 不可以。后者会出现循环位，存储空间有限时就必须舍入。不要把“十进制写得短”误认为“二进制也写得短”。

浮点数把小数点的位置交给一个指数记录。比如 $101.011_2 = 1.01011_2 \times 2^2$。向左移动小数点两位，就要乘 2^2 补回来。它与十进制科学计数法相似：有效数字决定精细程度，指数决定大致大小。

来源：课程第 3 讲 PDF 第 3—6、10—11 页；教材 2.4.1—2.4.2。

2. 存下来的 exp、frac，不等于公式里的 E、M

先统一符号。一个格式有 1 个符号位、k 个阶码位、n 个小数字段位；总长度为 $1 + k + n$。以下写 `exp`、`frac` 时指**字段的位串**，写 e、F 时指这两个字段按无符号数解释的整数。

“字段”就是完整位串中划出来的一段。例如一个8位教学格式规定 $k = 3, n = 4$，存储内容是 `0 | 101 | 0110`：符号占1位，`exp` 是3位的 `101`，按无符号整数读得 $e = 5$；`frac` 是4位的 `0110`，按无符号整数读得 $F = 6$。这里 k, n 是长度，e, F 是字段的整数值；它们不是额外存储的数据。小数部分要再算 $F/2^n = 6/16$，不能直接把 $F = 6$ 当成有效数 M。

初学补充：Q014：字段、位串、k/n、e/F分别是什么意思？ 用同一组8位数据区分存储内容、字段整数值和公式中的E、M。

图 D2–1　浮点位布局与解释路径

从字段到数值：先分类，再解释（1/3）

1 · 先读位串，不能直接代入数值公式

s	exp	frac
1 位符号	k 位 → 无符号整数 e	n 位 → 无符号整数 F

最高位在左　　最低位在右

exp 非全 0、非全 1	**规格化**
exp 全 0	**非规格化或零**
exp 全 1	**无穷或 NaN**

exp 与 frac 是存储字段；E 与 M 是按类别解释出的指数和有效数。

从字段到数值：先分类，再解释（2/3）

2 · 规格化：恢复隐藏的前导 1

条件：$1 \leqslant e \leqslant 2^k - 2$

阶码字段 → 真实指数

$$E = e - Bias$$

$Bias = 2^{k-1} - 1$
exp 用偏置编码，不是补码。

小数字段 → 有效数

$$M = 1 + F / 2^n$$

前导 1 不占 frac 存储位。
$M = 1.frac_2$

$$V = (-1)^s \times M \times 2^E$$

从字段到数值：先分类，再解释（3/3）

3 · 非规格化：固定指数，取消隐藏的 1

条件：$e = 0$，$F \neq 0$

真实指数

$$E = 1 - Bias$$

与最小规格化指数相同，
让零附近的刻度平滑衔接。

有效数

$$M = F / 2^n$$

这里是 $0.frac_2$，
不能补上规格化的前导 1。

e=0，F=0 → ±0　　**exp 全 1，F=0** → ±∞　　**exp 全 1，F≠0** → NaN

符号位最后决定正负。先确认 exp 的类别，才能选择正确的 E 与 M；无穷和 NaN 不套有限数公式。

图里最关键的是“先分类”。不能看到阶码就机械地减偏置，因为全 0 与全 1 的阶码是特殊情况。对于普通规格化数，M 还包含一个没有存进去的前导 1。

若还不清楚图在做什么：它是在把存储位还原成“正负号 × 有效数 × 2的指数次方”。规格化数把frac接在 `1.` 后面；非规格化数把frac接在 `0.` 后面，并固定使用最小规格化指数，以表示更靠近零的数。两者是按阶码选择的不同规则，不是先后执行的两步。Q014追问：这两张规格化与非规格化图到底在干什么？ 用8位格式完整计算

5.5和0.09375，并说明为什么要减偏置、补前导1或改用前导0。

偏置是 $\mathrm{Bias}=2^{k-1}-1$。例如 $k=4$ 时 Bias=7。存下的 $e=5$ 对应真实指数 $E=5-7=-2$；负指数并没有用补码保存。exp 本身是按无符号数读出的，随后才减偏置。

类型	字段条件	指数 E	有效数 M 或含义
规格化数	$1\le e\le 2^k-2$	$e-\mathrm{Bias}$	$1+F/2^n$，即 $1.\mathrm{frac}_2$
非规格化数	$e=0$，$F\neq 0$	$1-\mathrm{Bias}$	$F/2^n$，即 $0.\mathrm{frac}_2$
正零、负零	$e=0$，$F=0$	可沿用非规格化公式	$s=0$ 是 +0，$s=1$ 是 −0
正无穷、负无穷	$e=2^k-1$，$F=0$	不套有限数公式	符号由 s 决定
NaN	$e=2^k-1$，$F\neq 0$	不套有限数公式	非数；不是某个特别大的实数

为什么非规格化的指数是 $1-\mathrm{Bias}$，不是 $0-\mathrm{Bias}$？因为从最大的非规格化数走到最小的规格化数时，希望步长平滑衔接。这里既取消隐藏的 1，又保留最小规格化指数，使靠近零的数按等间距排列。代价是越靠近零，有效数字可能越少。

常见格式可以从总位数检查记忆是否正确：单精度 binary32 为 1+8+23=32，Bias=127；双精度 binary64 为 1+11+52=64，Bias=1023。规格化数的有效精度分别是 **24 位和 53 位**，因为还包含隐藏的 1。不要把“23 位 frac”说成“只有 23 位有效数字”。本讲默认课程使用的 IEEE 二进制格式与最近偶数舍入。

23和52首先来自格式规定的位数分配：binary32规定符号占1位、阶码占8位，所以frac剩下32−1−8=23位；binary64规定符号占1位、阶码占11位，所以frac剩下64−1−11=52位。只知道总长度并不能唯一推出字段长度。

随后才有24=1+23、53=1+52：数的是有效数 $1.\mathrm{frac}_2$ 中的二进制位，前导1虽不存储，仍计入有效精度；不包含符号位和阶码位。Q014追问：23/52位frac和24/53位有效精度分别怎么来的？ 区分格式规定的分配与隐藏位带来的有效精度。

来源：课程第 3 讲 PDF 第 11—22 页；教材 2.4.2—2.4.3；第 3 讲普通研讨题 1(3)—(7)。

3. 解码和编码都用固定步骤

解码时，先把十六进制完整展开为固定位数的二进制，按 $1/k/n$ 分段；再分类、求 E,M，最后才计算符号和数值。十六进制的一位是 4 个二进制位，但字段分界不一定落在十六进制数字边界上。

编码时，先分离正负并把绝对值写成二进制。如果它处于规格化范围，就化成 $1.xxx_2\times 2^E$，再算 $e=E+\mathrm{Bias}$；如果太小，就使用非规格化固定步长；如果位数太多，要舍入。**最后检查舍入是否导致进位、类型变化或溢出。**

例题 D2-E01：把往年题拆成可重复的动作

来源：2022 期中无答案版（来源资料未随仓库分发），第二题第 1 小题、PDF 第 8 页。题意截取：一个 8 位格式的符号、阶码、小数字段长度是 1、3、4；解码 0xBD，再编码 $9/64$。按该题的“类似 IEEE”前提使用上述特殊字段规则。

第一问：0xBD 是多少?

1. 展开并分段：`10111101 = 1 | 011 | 1101`。
2. $k=3$，所以 Bias=$2^2-1=3$。$e=3$，既非全 0 也非全 1，是规格化数。
3. $E=3-3=0$。$M=1.1101_2=1+1/2+1/4+1/16=29/16$。
4. 符号位为 1，所以 $V=-29/16=-1.8125$。

第二问：$9/64$ 怎么存?

此格式最小规格化正数是 $2^{1-3}=1/4$。$9/64<1/4$，不能硬塞进规格化公式。非规格化数的步长为 $2^{1-3-4}=1/64$，于是 $9/64$ 正好是 9 个步长，$F=9=1001_2$。符号为 0，阶码全 0，拼成 `0 | 000 | 1001 = 00001001`，即 `0x09`。

再检查：代回公式 $(9/16)\times 2^{-2}=9/64$，正好一致。编码完成后代回解码公式，是很便宜也很有效的自检。

4. 范围和精度：浮点数是一把间距会变的尺

增加阶码位，主要扩大数量级跨度；增加 frac 位，主要在已有数量级之间放入更多刻度。固定总位数时，两者会发生取舍，所以“位数一样多”不能推出“范围一样”或“精度一样”。

对一个 $1/k/n$ 格式，有以下几个常用边界：

问什么	公式	怎样得到
最小正非规格化数	$2^{1-\text{Bias}-n}$	exp 全 0，frac 只有最低位为 1
最大正非规格化数	$(1-2^{-n})2^{1-\text{Bias}}$	exp 全 0，frac 全 1
最小正规格化数	$2^{1-\text{Bias}}$	exp 的整数值为 1，frac 全 0
最大有限正数	$(2-2^{-n})2^{2^k-2-\text{Bias}}$	exp 是全 1 减 1，frac 全 1
同一规格化指数区间内的间距	2^{E-n}	frac 增加 1，数值增加 $2^{-n}2^E$

这里“最小正数”绝不包括 0；“最小负数”按数值大小通常指最负的那个。若题目说“最小负非规格化数”，必须看它是指最负的非规格化数，还是绝对值最小的负数，不能擅自混用。

图 D2–2　一个 7 位格式在零附近的刻度（助手补充）

设 $k=3, n=3$，Bias=3。以下只画非负部分：



这幅图说明两个事实：非规格化与规格化交界处没有突然留下一个大洞；到新的二进制数量级之后，刻度间距会加倍。负数部分按符号镜像分布，但 +0 和 −0 的位模式不同、数值比较相等。

单精度在 $[2^{23}, 2^{24})$ 中的间距为 1，在 $[2^{24}, 2^{25})$ 中的间距为 2。因此从 0 到 2^{24} 的整数都能精确表示，但 $2^{24}+1$ 不行，$2^{24}+2$ 又可以。**不能精确表示一个整数，不意味着更大的整数都不能表示。**

一般地，若指数范围足够，n 位 frac 的格式能连续精确表示到整数 2^{n+1}；第一个不能精确表示的正整数是 $2^{n+1}+1$。题目若问“能精确表示的最大整数”，则是另一个问题，需要看最大有限数。例如 D2–E01 的格式最大有限数只有 15.5，其最大可精确表示整数为 15，不能盲套连续整数公式。答题时先圈出“最大”“连续”“所有”这些词。

来源：课程第 3 讲 PDF 第 25—29、48 页；教材 2.4.3、练习题 2.49。图 D2–2 为助手重绘。

5. 舍入：先找相邻刻度，再判断中点

默认规则的完整名字是“舍入到最近值，中点时取偶数”。大多数时候只是选最近的刻度；只有恰好在中间，才检查保留部分的最低有效位。二进制的“偶数”是这个位为 0，**不是要求最终十进制数为偶整数**。

你可以用两种等价方法。第一种是在数轴上找上下两个相邻可表示数，比较距离；第二种是在二进制上保留 n 位小数，检查删掉的部分：

删掉部分	与半个间距比较	动作
`0xxxx...`	小于一半	保留部分不变
`1000...`	恰好一半	保留部分最低位为 0 则不变，为 1 则加 1
`1xxxx...` 且后面还有 1	大于一半	保留部分加 1

这里“加 1”是给保留部分的末位加一个单位，可能连续进位。负数同样比较距离；遇到中点，也让保留后的最低有效位是 0。不要把“负数都向零靠”当成最近偶数舍入，那是另一种模式。

Q015追问：D2–E03的最近偶数判断，就是表里的1000…这一行吗？ 将精确结果切成保留的23个小数位和舍弃的 `1000…`，展示连续进位及恢复负号的过程。

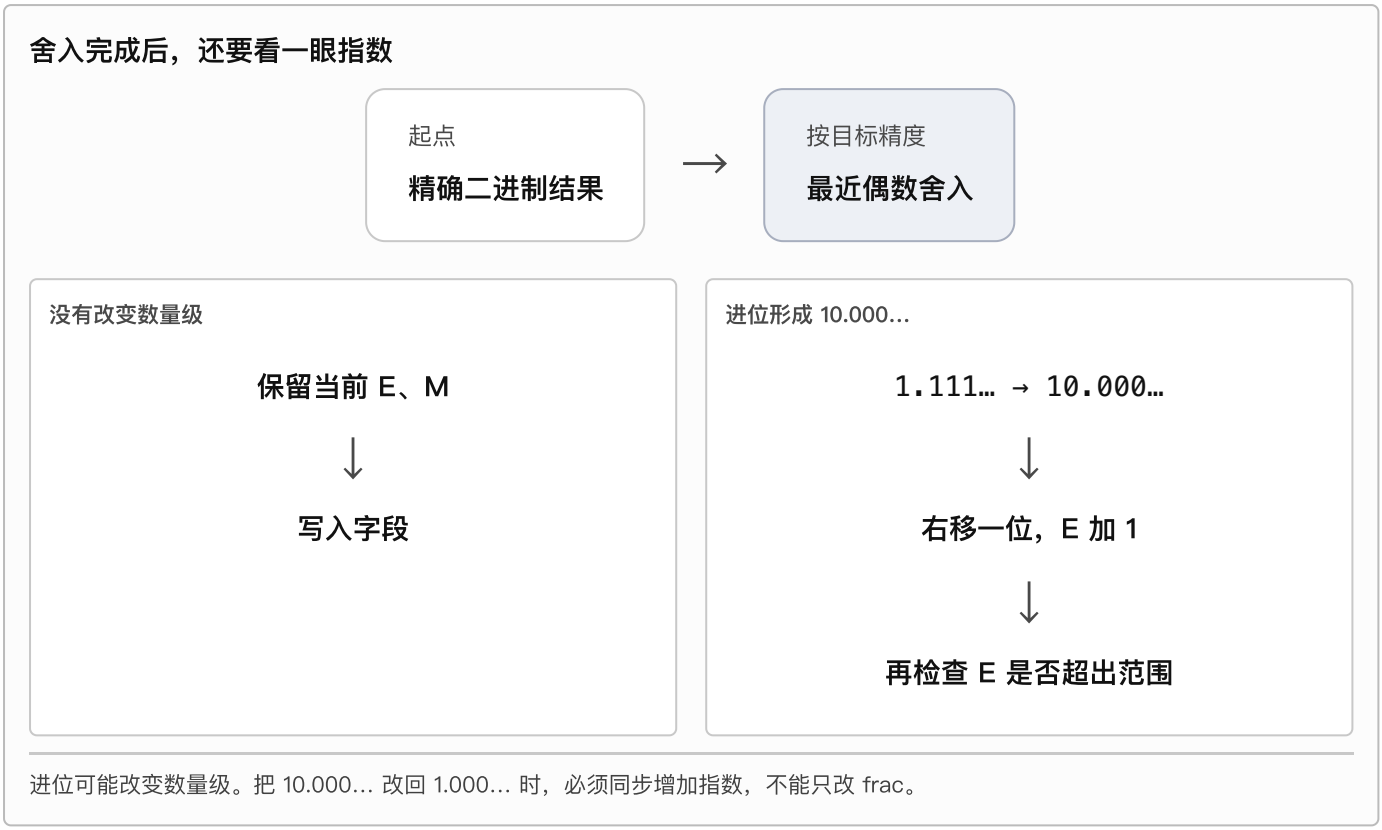
例题 D2–E02：中点舍入为什么还要改指数?

助手补充，格式为 8 位 `1/4/3`，Bias=7。把 13.5 和 15.5 编码，均用最近偶数舍入。

对 $13.5 = 1101.1_2 = 1.1011_2 \times 2^3$，只能保留 `1.101`。删掉的是 `1000...`，恰好在 13 和 14 中间；保留末位是 1，必须进位到 `1.110`。指数仍为 3，存储阶码为 10，即 `1010`，得到 `0 | 1010 | 110 = 0x56`，数值为 14。

对 $15.5 = 1111.1_2 = 1.1111_2 \times 2^3$，仍是中点，末位 1 要进位。可是 `1.111 + 0.001 = 10.000`，已经不符合规格化的 `1.xxx` 形式。要改写为 1.000×2^4。因此新 $e = 4+7 = 11$，得到 `0 | 1011 | 000 = 0x58`，数值为 16。

图 D2–3　舍入后的检查流程



图中最后一次检查不能省。如果重新规格化后已经没有可用有限阶码，在最近偶数模式下可能得到无穷。很小的数也会在舍入后变成零，或从非规格化跨入最小规格化数。并不是所有大于最大有限数的实数都立刻舍成无穷；靠近上边界时仍应按目标格式的舍入规则判断。

若看到课件中的 G、R、S，不必把它当成另一套原理。第 3 讲第 46 页把 **G 定为最后保留的位，R 定为第一位被删的位，S 定为后续所有被删位的按位或**。在这一命名下，进位条件是 $R \wedge (S \vee G)$：R=0 不到一半；R=1 且 S=1 超过一半；R=1、S=0 恰好一半，此时由 G 决定。其他材料可能把 G 命名在第一位被删的位置，做题一定先读定义，别只记三个字母。

材料校验：第 3 讲 PDF 第 47 页有数值笔误，例如 $\texttt{1.001}_2 \times 2^7$ 写成 134，实际是 144。本讲重新计算所用例题，不照抄该页结果。

来源：课程第 3 讲 PDF 第 31—36、44—47 页；教材 2.4.4—2.4.5；第 3 讲普通研讨题 2(1)—(4)。

6. 浮点加乘与 C 转换：每次运算都可能丢信息

本阶段把浮点运算理解为“先求精确结果，再舍入到目标格式”就够了。加法先对齐二进制小数点，再按符号加减、规格化和舍入；乘法先异或符号位、相乘有效数、相加指数，再规格化和舍入。乘法的指数是**真实指数 E 相加**，不能直接把两个带偏置的 exp 字段相加当作答案。

例如在 `1/4/3` 格式中，$1.5 \times 1.75 = 2.625 = 1.0101_2 \times 2^1$，位于 2.5 和 2.75 中间。保留 `1.010` 的末位为 0，所以存成 2.5。算浮点乘法时只写精确积 2.625，会漏掉最后一层舍入。

例题 D2–E03：为什么不能任意移动括号?

助手补充。假设每一步都按 binary32 运算并舍入，令 $a = 2^{25}$、$b = -2^{25}$、$c = 1$。

第一种括号 $(a + b) + c$：先精确抵消得 0，再加 1，结果为 1。第二种括号 $a + (b + c)$：先算 $-2^{25} + 1$，它在相邻可表示值 -2^{25} 与 $-2^{25} + 2$ 的中间，最近偶数规则选择 -2^{25}，再与 a 相加得 0。

展开看：精确结果是−33,554,431，左右可表示值是−33,554,432和−33,554,430，各差1。中点取偶数看的是有效数最低保留位，不是十进制整数奇偶；前一候选的frac全0，后一候选的frac全1，所以选前者。Q015：为什么-2^{25}加1会先舍回-2^{25}，最后得到0？ 按“精确求和→找相邻值→判断末位→再加a”逐步展开。

两种写法的实数结果相同，浮点结果却不同。反例的要点是每一步明确舍入，不能仅说“大数吞小数”。特别要注意二次幂附近的刻度：某个数左右两边的间距可能不同。找中点时应把真正相邻的两个可表示数写出来，不要看见量级很大就断言加 1 一定消失。

浮点加法、乘法一般不满足结合律，乘法对加法一般也不满足分配律。对于有限且不产生 NaN 的相应运算，非严格的单调关系仍有用；严格大于可能在舍入后变成相等。遇到无穷和 NaN，要另查特殊规则，不能把实数代数直接搬过来。

以下转换规则默认 `int` 是 32 位，`float/double` 分别为 binary32/binary64：

这里“转换”是把一个数值从一种数据类型转换成另一种类型。例如 `int x=3; float y=x;` 会读取整数3，把它转换成float的3.0并存入y，x仍是int且值不变；`int → float` 只是转换方向的写法，不是C运算符。表格关心的是转换后能否保留原值，还是必须舍入、去掉小数部分或遇到范围限制。Q016：这张表里的类型转换是什么意思？ 用实际C代码说明自动转换、显式转换及与原位串重新解释的区别。

转换	应当先想到什么
`int → float`	范围足够，但 24 位精度可能不够，可能舍入
`int → double`	53 位精度足以精确表示所有 32 位 int
有限 `float → double`	原来的数值可以精确保留，不会补回以前已经丢掉的信息
`double → float`	可能舍入，也可能超出 float 范围；课程 IEEE 模型下可能得到无穷
有限 `float/double → int`	去掉小数部分，向零截断；还要检查截断结果能否由 int 表示

例如 `(int)-3.9` 是 −3，不是 −4；这与浮点算术默认的最近偶数舍入不是一回事。NaN、无穷或截断后超出整数范围的浮点到整数转换，不能当成可移植 C 的正常数值运算；不要无条件回答“必为 TMin”。具体指令可能产生某种硬件结果，那需要题面提供机器前提。

还要检查表达式的类型：`2/3` 先做整数除法，结果为 0；`2/3.0` 做浮点除法。赋值给 `double` 并不能倒回去把已经完成的整数除法重新算一遍。

7. 零、无穷、NaN 与“看位”

在课程采用的 IEEE 语义下，+0 与 −0 的位不同，但 `+0.0 == -0.0` 为真；其符号在倒数等运算中仍可观察，例如正数除以 +0 与 −0 会给出不同符号的无穷。NaN 更特殊：`NaN == NaN` 为假，`NaN != NaN` 为真，与 NaN 做 <、<=、>、>= 都为假。不能用 `x == NaN` 判断是不是 NaN，应按题意使用 `isnan`，或在明确 IEEE 前提时利用 `x != x`。

“转换成整数”与“把相同位串看成整数”是两件不同的事。前者尽量保留数值，改变编码；后者保留编码，改变解释。比如 binary32 的 1.0 编码是 `0x3f800000`，数值转换成 int 得 1；把其位串读作 32 位整数则得到 1065353216。后者不是 `(int)1.0f` 的含义。如何在 C 中安全操作这些字节，会在联合与内存一讲继续解释。

例题 D2-E04：往年题中的“半字节浮点”

来源：2024 期末无答案版（来源资料未随仓库分发），第一题第 1 小题、PDF 第 2 页。按题意截取并改为表格分析：`float4` 有 1 个符号位、2 个阶码位、1 个 frac 位；`int4` 是 4 位补码。`f2b` 保留位串后按 int4 解释；`f2i` 把浮点值向零转换为 int4。原题用假想类型说明规则，不是能直接编译的标准 C 类型。

判断候选位串 `0000`、`0001`、`1000`、`1101` 中，哪些使二者相等。

先读懂任务：每个候选给的是原始4位，不是已经确定的整数值。对同一个候选，分别走两条独立路线：**f2b直接把原位串按4位补码读；f2i先按float4解出浮点值，再向零取整。** 比较的是两条路线最后得到的整数，不是让一个函数接着另一个函数执行。例如 `0001` 直接按补码读是1，先按浮点读却是0.5，再取整得到0，所以不相等。Q017：半字节浮点题到底给了什么、要比较什么？ 逐项解释两种类型、两条路线和四个候选。

Bias=$2^1-1=1$。逐个按 `s|exp|frac` 分段：

位串	浮点值	同位串的 int4 值，即 f2b	向零转换的 f2i	是否相等
`0|00|0`	+0	0	0	是
`0|00|1`	$0.1_2\times 2^0=1/2$	1	0	否
`1|00|0`	−0	−8	0	否
`1|10|1`	$-1.1_2\times 2^1=-3$	−3	−3	是

故相等的是 `0000`、`1101`，对应原题选项 D。最容易错的是把 −0 的位串当成整数的 0，以及把 `0001` 在两种格式下都解释成 1。题目在考解释规则，不能只看位串长得像哪个熟悉的数字。

8. 条件码：CPU 记下的四个一位结果

汇编没有直接写出的 `if (a<b)`。常见实现是先计算一次比较、保存几个标志，再让跳转或赋值指令查看标志。四个标志是：

标志	问的问题	初学时的检查方法
ZF	固定位宽结果是否全 0?	全 0 为 1，否则 0
SF	结果最高位是否为 1?	直接抄结果最高位
CF	无符号加法是否进位 / 减法是否借位?	加法超出无符号范围；减法时无符号被减数小于减数
OF	补码意义下是否溢出?	加法同号输入变异号；减法异号输入且结果符号异于被减数

先用4位教学运算理解：无符号范围是0～15，补码范围是−8～7。`0111+0001=1000` 按无符号读是7+1=8，没有向第5位进位，CF=0；按补码读却把本应为8的结果存成−8，超出有符号范围，OF=1。对于减法A−B，A叫被减数、B叫减数；CF检查无符号A是否小于B，OF检查补码意义下的精确差是否越界。Q018：CF的进位/借位与OF的有符号溢出是什么意思？ 逐位展开加法与减法，并解释表中的符号判断。

ZF、SF 只需看截断后的结果。CF、OF 要结合运算与输入；它们互不等价。一次运算可以只有 CF=1、只有 OF=1、两者都为 1，或都为 0。SF=1 也不能说明“算错了”或“发生溢出了”，它只描述结果位串的最高位。

AT&T 语法遵循“源在前、目的在后”。因此 `subq b,a` 做 $a\leftarrow a-b$；`cmpq b,a` 也按 **a−b** 设置标志，但不把差写回 a。记成“比较后一个与前一个”，然后在草稿上写出差，就不容易反过来。

`testq b,a` 根据 `a & b` 设置标志，也不改写操作数。它把 CF、OF 清零，ZF、SF 看按位与结果。`testq %rax,%rax` 实际检查 rax 是否为零、符号位是否为 1；`testb $1,%al` 检查最低位是否为 1，常用于奇偶判断。前者不是真的计算平方，后者也不是真的比较 al 与 1。

图 D2–4　比较、标志与三类使用方式

先用“是否相等”读这张图。假设rax和rbx都为5，执行 `cmpq %rbx, %rax`，CPU按5−5的结果设置标志：ZF=1，但rax、rbx仍为5。后面的指令不重新比较这两个寄存器，而是查看已保存的ZF。图中的 `cc` 是“条件后缀”的占位符，例如换成 `e` 表示相等，此时检查ZF=1。



下面三行分别说明图中的三种用法，每行都独立接在上述比较后理解，不要求按顺序执行：

实际指令	条件成立（ZF=1）时	条件不成立（ZF=0）时
`sete %cl`	把cl这个字节写成1	把cl写成0
`je .L_equal`	跳到标签.L_equal标出的代码位置	继续执行紧接着的下一条指令
`cmove %rsi, %rdx`	把rsi的值复制到rdx	rdx保持原值

因此setcc是把判断结果存成0或1，jcc是选择接下来执行的位置，cmovcc是有条件地复制数据。标签只是代码位置的名字；“写一个字节”也不是“只写一个位”。Q019：cmp之后的setcc、jcc、cmovcc到底分别在做什么？从5与5的比较出发，并对照mov保留标志、add覆盖标志的过程。

读图时注意，条件指令读取的是**所需标志当前保存的值**，要追踪每个相关标志最近由谁修改，不是固定读取上方最近的cmp。`mov`、`lea`、`setcc`、`jcc`、`cmovcc` 不会重新设置这四个标志；`add/sub` 会，逻辑运算会。`inc/dec` 会修改ZF、SF、OF，但保留CF。例如比较相等后ZF=1，中间的mov不会改变它；若再执行一次结果非零的add，ZF就改成0，随后je不再跳转。

不要把移位指令笼统记成“OF 一律清零”。真实 x86 的移位标志与指令及有效移位次数有关：次数为 0 时不改标志；单个位移的 OF 规则依指令而异，多位移的 OF 通常未定义。本讲练习不靠未定义的 OF 出题。教材 3.6.1 对移位 OF 的一句简化表述不能当作所有移位的通用规则。

来源：课程第 5 讲 PDF 第 13—25 页；教材 3.6.1—3.6.2；第 5 讲普通研讨题 1(1)—(5)。

9. 同一次 cmp，为什么有符号和无符号会走不同分支？

`cmp` 自己没有 signed 或 unsigned 版本：同样宽度的减法产生同一组标志。区别在后面选择哪个条件。下表按刚刚执行的 `cmp b,a` 来读，“a<b”中的 a 是被减数。

关系	signed 后缀	条件	unsigned 后缀	条件
a=b	e 或 z	ZF	同左	同左
a≠b	ne 或 nz	非 ZF	同左	同左
a<b	l	SF≠OF	b	CF
a≤b	le	(SF≠OF) 或 ZF	be	CF 或 ZF
a>b	g	(SF=OF) 且非 ZF	a	非 CF 且非 ZF
a≥b	ge	SF=OF	ae	非 CF

后缀可以接在 `set`、`j`、`cmov` 后面。例如 `setl %al` 把有符号小于关系写成 0/1，`jl .Lsmall` 在同一条件下跳转。这里 `setl` 的 l 是 less，不是 32 位 long 后缀；所有 setcc 都只写一个字节。

这些是本讲需要会读的常用指令，先按“动作＋条件”拆开，不把每种组合当成全新的命令背。`set+l` 是“满足有符号小于条件就写1，否则写0”；`j+l` 是“满足同一条件就跳转，否则继续下一条”；`cmov+l` 是“满足同一条件就复制，否则保留目的原值”。例如rdi=3、rsi=5，先执行 `cmpq %rsi,%rdi`，再执行 `setl %al` 得到al=1；若改接 `jl .Lsmall`，则跳到标签.Lsmall。条件指令只读取已有标志，并不重新比较两个操作数。

注意区分指令类别：`movl/addl/cmpl` 的l表示32位宽度；`setl/jl/cmovl` 的l表示有符号less条件。`setl %al` 只写8位al，不会把整个rax都写成0或1。第10节再解释怎样整理高位。Q019追问：这些指令需要掌握吗，setl和jl该怎样拆开读？ 给出本讲练习所需的学习范围与条件后缀速查，不代表完整考试指令清单。

为什么有符号小于使用 SF 异或 OF，而不直接看 SF？没有溢出时，差为负当然意味着 a<b；有溢出时，补码结果的符号反了，需要 OF 把判断再翻一次。CF 则保留无符号减法是否需要借位的信息，所以无符号比较只需另选对应条件。

记忆时先抓三条：相等看ZF=1，无符号小于看CF=1，有符号小于看SF≠OF。再由“小于等于＝小于或相等”“大于等于＝不是小于”“大于＝不是小于且不相等”推其余条件。Q019追问：条件表可以直接背吗，怎样少记几条？说明推导方法与比较顺序、标志未被覆盖的前提。

例题 D2-E05：SF 为 1，也可能是 a>b

助手补充。8 位寄存器 al 初值是 `0x64`，执行 `cmpb $0xd8,%al`。

先按无符号整数读操作数：`0x64=100`、`0xd8=216`，精确差是−116。只保留低8位，相当于取模256：−116+256=140，而140=8×16+12，所以结果位串是 `0x8c`，即 `1000 1100`。因此这里的 `0x64-0xd8 → 0x8c` 表示固定8位运算的结果，不是普通整数等式。al仍是 `0x64`，因为cmp不写回。Q020：0x64减0xd8，为什么低8位是0x8c？ 用十进制换算、十六进制借位及两种数值解释逐步核对。

1. 差非零，ZF=0；最高位为 1，SF=1。
2. 按无符号解释，100−216 需要借位，CF=1。
3. 按补码解释，`0xd8` 是 −40，真实差是 100−(−40)=140，超过 8 位补码最大值 127，OF=1。
4. signed 小于条件是 SF≠OF，这里不成立；signed 大于条件成立。因此 `jg` 会跳，符合 100>−40。
5. unsigned 小于条件是 CF=1，所以 `jb` 会跳，符合 100<216。

同一条 cmp、同一组标志，两种判断都正确，因为问的是两种不同解释下的关系。

10. setcc 写一字节，返回整数还需整理高位

考虑返回 `x < y` 的函数，其中 x、y 是 64 位有符号 long，分别在 rdi、rsi：

```
cmpq    %rsi, %rdi       # 比较 x 与 y：概念差 x-y
setl    %al              # 只把 al 写成 0 或 1
movzbl  %al, %eax        # 字节零扩展到 32 位；写 eax 又清零 rax 高32位
ret
```

假设原 rax 是 `0xffffffffffffffff`，比较结果为假。只执行 `setl %al` 后，rax 是 `0xffffffffffffff00`，还不是整数 0。执行 `movzbl` 后才成为全 0。比较结果为真时，则整理成完整的整数 1。

`movzbl` 名字中的 b 和 l 表示从 8 位扩展到 32 位，不是两个不同的条件。写 `%al` 不会清除其余位；写 `%eax` 会自动清除 `%rax` 高 32 位。也不要在 `cmp` 与 `setcc` 之间随手插入 `xor %eax,%eax` 清零，它会改写条件码，破坏原比较结果。若要先清零，应放在比较之前。

11. if 与 cmov：先说明执行了什么，再谈结果

例题 D2-E06：把有符号最大值写成两种实现

助手补充。函数返回两个 long 中较大者，不涉及加减溢出：

```
long larger(long x, long y) {
    if (x > y) return x;
    return y;
}
```

一份便于阅读的分支实现是：

```
cmpq    %rsi, %rdi       # x-y；检查 x>y
jg      .Lx
movq    %rsi, %rax       # 未跳：x<=y，取 y
ret
.Lx:
movq    %rdi, %rax       # 已跳：x>y，取 x
ret
```

取 x=−2、y=3。比较后 `jg` 不满足，程序顺序执行 `movq %rsi,%rax`，返回 3；它不会又走到 `.Lx`，因为已经 ret。读分支题时，应沿一条真实路径走，不要把两个分支的赋值都叠加起来。

同一函数也可写为：

```
movq    %rsi, %rax       # 先准备候选 y
cmpq    %rsi, %rdi
cmovg   %rdi, %rax       # 若 x>y，用 x 替换；否则保持 y
ret
```

`cmovg` 不跳转，它只决定复制不复制。两份代码都返回最大值，但这不意味着任意 if 都可以把两边提前计算。以下是普通研讨题要求理解的三类限制：

- **可能出错的表达式。** `p ? *p : 0` 在 p 为空时本来不访问 `*p`。若先加载 `*p` 再用 cmov 选 0，访问错误已经发生；结果最终选 0 也救不了。
- **带副作用的表达式。** 只有选中的分支应修改变量、写内存或调用带副作用的函数，不能把两边都执行一遍再只挑结果。
- **计算代价很大的表达式。** 两边都计算可能浪费大量工作，即使语义允许，也未必更快。

特别地，条件传送的内存源操作数也不能被想象成“条件为假就一定不访存”。做安全判断，应先检查候选值的获取是否可能触发错误。上述空指针分支可以使用正常跳转；更复杂的安全改写需要另证正确性，本轮不为它扩展新技巧。

读短汇编时在每个分支前写四项：当前类型/位宽、比较的真实顺序、跳转条件、跳与不跳各到哪里。循环与 switch 会在下一讲建立在这个流程上。

来源：课程第 5 讲 PDF 第 27—35 页；教材 3.6.3、3.6.5—3.6.6；第 5 讲普通研讨题 1(6)—(7)、1(9)。不布置其中的拓展研讨。

12. 独立练习：先交过程，再核对答案

以下全部为助手补充的独立训练题，编号固定为 D2-P01—D2-P10。必做 8 题共 30 分：4 道浮点短题、3 道条件短题、1 道短分支对照题。它们不是额外的一套整卷，也不是用户已完成的作答记录。

默认 x86-64、Linux、AT&T 语法；所有浮点题用题面给定的 IEEE 式字段规则和最近偶数舍入；不讨论信号 NaN 的异常行为。写出字段、公式或比较关系，仅写最后一个数不足以定位错误。

D2-P01　解码两个不同类别的数（4 分）

8 位浮点格式为 `1/4/3`。分别解码 `0x25` 和 `0x83`。每个数写出分段、类别、E, M 和精确数值，数值可写成分数。

D2-P02　范围与特殊值（4 分）

7 位格式为 `1/3/3`。求最小正非规格化数、最小正规格化数、最大有限正数；再判断 `0 | 111 | 010` 表示什么。每项 1 分，边界题必须写出依据。

D2-P03　中点与重新规格化（4 分）

将 $21/4$ 与 $31/8$ 编码成 `1/4/3` 的 8 位浮点数。每问写出精确二进制形式、舍入方向与理由、舍入后数值和十六进制编码。

D2-P04　数值转换（3 分）

假设 int 为 32 位，float/double 为 binary32/binary64，使用默认最近偶数舍入。

1. 对所有 int `x`，`x == (int)(double)x` 是否成立？说明原因。
2. 若 `int x = 16777219;`，求 `(int)(float)x`，说明邻近刻度与舍入理由。
3. 求 `(int)-6.75`，说明使用哪一种取整方向。

第2小题追问：Q021：16777219在两个刻度中间，为什么选16777220？（含该小题解答；展开最低保留位、刻度单位与中点进位，按需打开）。

D2-P05　分别计算 CF 和 OF（4 分）

下面两段独立执行。分别给出最后的 al 与四个标志 CF、ZF、SF、OF。

```
# A
movb $0x7f, %al
addb $1, %al

# B
movb $0x00, %al
subb $1, %al
```

D2-P06　选择 signed 与 unsigned 条件（3 分）

设 edi=`0xffffffff`，esi=`0x00000001`。执行 `cmpl %esi,%edi` 后，分别求 `setl %al` 与 `setb %al` 会写入的值，并说明二者为何不同。两条 set 指令均依据同一次 cmp 的标志，不要把其中一次 set 当成新的比较。

D2-P07　部分寄存器与条件码保持（3 分）

初始 rax=`0xaabbccddeeff0011`，edi=0。依次执行：

```
testl   %edi, %edi
setne   %al
movzbl  %al, %eax
```

写出执行 setne 后的完整 rax、执行 movzbl 后的完整 rax，以及最终 CF、ZF、SF、OF。说明为什么不能把 setne 后的 rax 直接当作布尔返回值。

D2-P08　短分支的 C 与汇编对照（5 分）

参数 x、y 是 signed long，分别位于 rdi、rsi；假设本题实际执行的加减结果都在 long 范围内。

```
choose:
    cmpq %rsi, %rdi
    jg   .Lelse
    leaq 2(%rdi), %rax
    ret
.Lelse:
    leaq -1(%rsi), %rax
    ret
```

1. 补全 C：`if (_____) return x+2; else return y-1;`。
2. 分别计算输入 `(x,y)=(-3,2)` 与 `(7,2)` 的返回值。
3. 若改为 unsigned long 比较、仍保持同样的“小于等于时走第一分支”意图，`jg` 应换成什么？
4. 用一句具体理由说明：能否把任意 `p ? *p : 0` 都改成“先读 `*p`，再条件传送”？

D2-P09　选做：同样位数，为什么有限实数数量不同？

两个 7 位格式，A 为 `1/3/3`，B 为 `1/4/2`。分别求它们表示的**不同有限实数**的数量，+0 与 −0 只算一个，NaN 和无穷不算。然后比较最小正非规格化数。不要只根据“B 指数多一位”猜数量；列出允许的 exp、frac、符号组合。

D2-P10　选做：更大不代表转换后仍严格更大

令 int a=16777216，int b=16777217。先比较 a、b；再求 binary32 中 `(float)a`、`(float)b`，判断它们之间的 `<` 和 `==`。若先分别转换成 double 再比较，结论如何？解释这是否反驳了舍入的**非严格**单调性。

13. 过关检查与资料定位

完成必做题后按答案中的作答点自评，24/30 是本讲首次独立练习的门槛。即使总分达标，若仍分不清 exp 与 E，或者把 `cmp b,a` 反读为 b−a，先修这两个前置错误再学循环。没达到门槛时，不重抄整章：把错误归为“分段/分类”“舍入”“操作数顺序”“条件类型”“部分寄存器”之一，每类重做一个更小例子，再做未见变式。这里不预填你的分数或掌握状态。

合上讲义，检查自己能否说清：非规格化为什么不用隐藏 1；最近偶数只在什么情况下看末位；浮点转 int 为何不使用相同舍入规则；为什么 cmp 不改寄存器却会影响跳转；为什么 SF=1 不能单独证明 a<b；为什么 setcc 后常跟 movzbl。能说清再做对题，才算把规则接到了操作上。

本讲使用的文件与精确定位如下。下列 PDF 页序从文件第一页开始；教材只引用已核对的小节，不把目录页序当印刷页码。

- 课程第 3 讲：浮点（来源资料未随仓库分发）：PDF 第 3—6、10—42、44—48 页。
- 课程第 5 讲：控制（来源资料未随仓库分发）：PDF 第 13—35 页；循环和 switch 留到第 3 讲复习。
- 教材《深入理解计算机系统》第 3 版（来源资料未随仓库分发）：2.4.1—2.4.6、3.6.1—3.6.3、3.6.5—3.6.6；部分寄存器规则参见 3.4.2。当前本地文件已核对 2.4.4—2.4.6 为印刷页 83—86、PDF 第 119—122 页，3.6.1—3.6.2 的相关说明为印刷页 135—137、PDF 第 171—173 页，3.6.6 为印刷页 145—148、PDF 第 181—184 页。
- 第 3 讲研讨题（来源资料未随仓库分发）：PDF 第 1—2 页的普通研讨要求；排除明确标为拓展的事故调研、NaN 载荷与阿贝尔群等题。
- 第 5 讲研讨题（来源资料未随仓库分发）：PDF 第 1 页“条件码和条件分支”普通题；本讲不要求流水线性能调研。

例题与独立题属于教学材料，后续你的真实提问和追问另按稳定编号记录，不把这些预先写成你已经提出的问题。

第 3 讲：循环、跳转表与过程调用

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

1. 读控制流，先找“下一步去哪里”
2. switch：先找入口，再沿代码走
3. 调用：把“去哪里”和“带什么”分开
4. ABI：函数之间约定怎么交接
5. 递归：重复的是代码，分开保存的是每次调用的状态
6. 遇到长题时，按这个顺序落笔
7. 独立练习：4 道必做，共 40 分
8. 本讲完成时应留下什么

来源与范围核对

本讲对应复习的第 3 天，主要覆盖课程第 5 讲的循环与 switch，以及课程第 6 讲的过程调用。今天要建立两种能力：看到一段反复跳转的代码，能还原“什么在重复、何时停止”；看到函数调用，能画出“控制去了哪里、什么值必须留下、返回后从哪里继续”。不用背长汇编，必须能追踪状态。

先读循环与跳转表，再读调用与递归。每部分的示范例题有完整推理；最后的四道必做题请独立作答后再看练习答案。必做总分 40 分，32 分为本日基础过关线，另有一道选做题。今天的任务不含 2025 年第一次小测原题。

统一前提。 以下采用 x86-64、Linux、System V AMD64 ABI 和 AT&T 汇编语法，`long` 与指针均为 8 字节。除专门讨论无符号索引外，C 中的 `long` 为有符号整数。示例给出有限输入范围，避免有符号溢出、除零等未定义行为。汇编是为讲解整理的等价实现，不声称某个编译器版本一定逐字生成它。不能在 Apple Silicon 或 Windows 的本机汇编上直接套用本讲的调用规则。

1. 读控制流，先找“下一步去哪里”

普通指令执行完，控制流通常走到下一条指令。`jmp .Lx` 改成无条件去标签 `.Lx`；`jl .Lx` 则只有条件成立时才去那里，否则继续向下。标签是代码位置的名字，不是变量。标签之间没有“隔墙”：如果前面没有跳走或返回，就会继续执行下一个标签后的代码。

读汇编时先复习三个最容易混淆的写法：`movq %rsi,%rax` 把值从 `%rsi` 复制到 `%rax`；`addq %rdx,%rax` 把 `%rax` 改成原来的 `%rax+%rdx`；`cmpq %rdi,%rdx` 按 `%rdx-%rdi` 的结果设置条件码，却不修改两个操作数。因此，后接 `jl` 表示按有符号数比较 `%rdx<%rdi`，不是反过来。

一段循环最值得先标出来的是四件事：**初始化、循环条件、循环体、更新**。初始化只在进入循环前执行一次；循环体是希望重复的工作；更新让状态向结束条件推进；条件决定下一轮是否执行。代码中这四件事可能被拆到不同位置，不能因为条件写在汇编末尾就认定原来是 `do-while`。

1.1 三种 C 循环，区别在第一次检查发生在哪里

`while` 先检查条件，再决定是否执行循环体。因此，它可能一次也不执行：

```
while (x > 0) {
    work(x);
    x--;
}
```

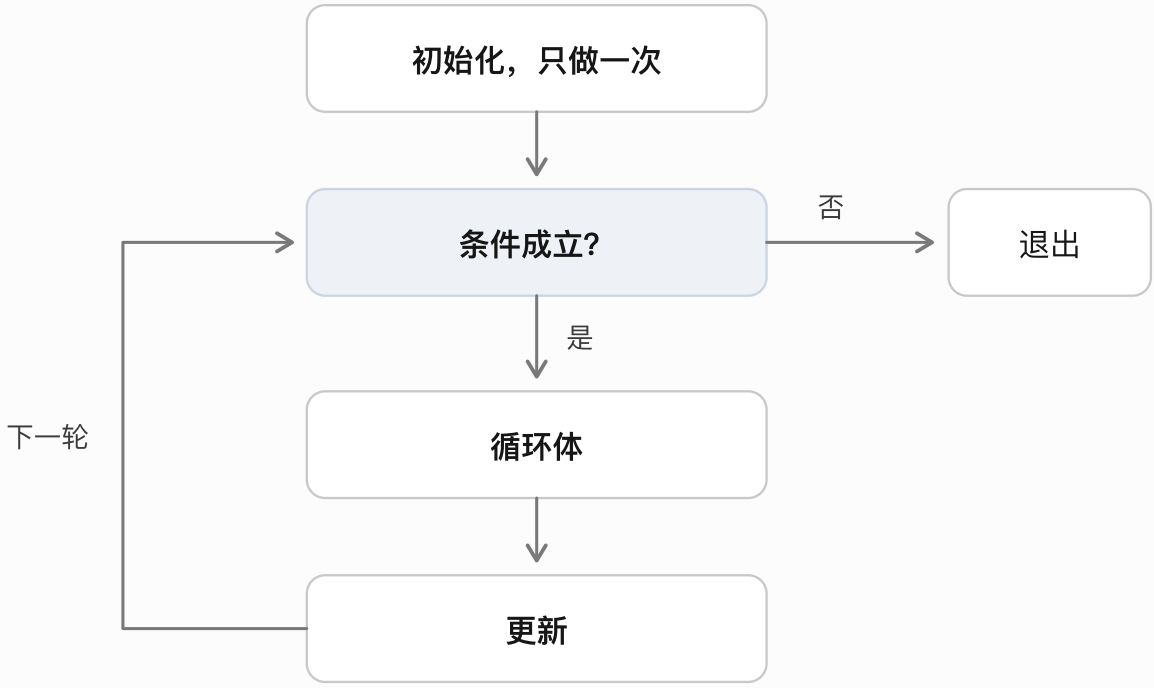
这里若开始时 `x==0`，就直接结束。`do-while` 先执行一次循环体，再检查条件：

```
do {
    work(x);
    x--;
} while (x > 0);
```

无论起始的 x 是什么，第一次 work(x) 都会发生。考试中，零值输入经常就是区分两种结构的最小反例。

for（初始化；条件；更新）循环体 的执行顺序是：初始化；检查条件；循环体；更新；再次检查条件。省略条件相当于条件始终为真。先把下面的图读懂，再去看汇编中这些部分摆在哪里。

循环执行的顺序



沿箭头走一轮：初始化没有回边；循环体执行完后，先更新，再回到条件。

读图时沿箭头走：初始化没有回来的箭头，所以只执行一次；“更新→条件”的返回箭头形成循环。若循环体执行 break，应直接去“退出”；若 for 的循环体执行 continue，应先去“更新”，再检查条件。直接把 for 改成 while 而忘了 continue 的去向，会改变程序行为。

1.2 条件在汇编末尾，也可能是 while

编译器可以先跳到末尾检查条件，满足后再回到前面的循环体。这叫“先跳到测试位置”的布局：

```
初始化
jmp .Ltest          # 第一次先判断，不先执行循环体
.Lbody:
    循环体
    更新
.Ltest:
    比较
    条件成立则跳回 .Lbody
结束
```

另一个常见布局是先在入口检查一次，不满足就结束；接下来循环体放在前面，循环条件放在后面。两种布局都能实现 while。所以还原 C 时，不必猜编译器原来用了哪个关键字；先写出行为等价、边界正确的循环。

所谓“回边”，就是从后面的代码跳回前面的代码。它能帮助定位循环，但不是完整答案。必须继续问：第一次如何到达这里？哪个寄存器变化？比较发生在更新前还是后？等号算不算继续？这些问题决定会不会少一轮、多一轮，或在输入零时错误地进入循环。

示例 D3-E01：从汇编恢复一个求和循环（助手补充）

输入 `0 <= n <= 1000`，`n` 在 `%rdi`，返回 `long`。给定：

```
sum_before_n:
    movl $0, %eax
    movl $0, %edx
    jmp .Lsum_test
.Lsum_body:
    addq %rdx, %rax
    addq $1, %rdx
.Lsum_test:
    cmpq %rdi, %rdx
    jl .Lsum_body
    ret
```

第一步，给寄存器起临时变量名。 `%rax` 初始为零，在每一轮累加 `%rdx`，最后又是返回寄存器，很像累计值 `sum`。`%rdx` 初始为零，每轮加一，很像计数器 `i`。给它们起名字只是帮助理解，最终还要靠整段代码验证。

第二步，翻译比较。 `cmpq %rdi,%rdx` 比较 `i-n`；`jl` 在 `i<n` 时进入循环体。这是有符号比较，与题目中的 `long` 一致。最前面的 `jmp .Lsum_test` 说明先判断，再执行循环体。

第三步，拼回 C。

```
long sum_before_n(long n) {
    long sum = 0;
    for (long i = 0; i < n; i++) {
        sum += i;
    }
    return sum;
}
```

第四步，用小输入检查。 `n=4` 时，四轮分别把 `0`、`1`、`2`、`3` 加进去，返回 `6`。结束时 `i==4`，这一轮只检查条件，不再加 `4`。`n=0` 时第一次条件就不成立，返回 `0`；这验证了入口跳转不能省略。这里对 `%eax`、`%edx` 写零会把整个对应 64 位寄存器的高半部分也清零，因此确实初始化了 `%rax` 与 `%rdx`。

可以再用一句话检查整个循环：每次到达 `.Lsum_test` 时，`sum` 等于 `0` 到 `i-1` 的和。初始 `i=0` 时是空和零；循环体加入当前 `i` 后再加一，下一次检查时这句话仍成立。这样的句子叫循环不变式，它比“看起来像求和”更能保证你没有漏掉最后一次迭代。

2. switch：先找入口，再沿代码走

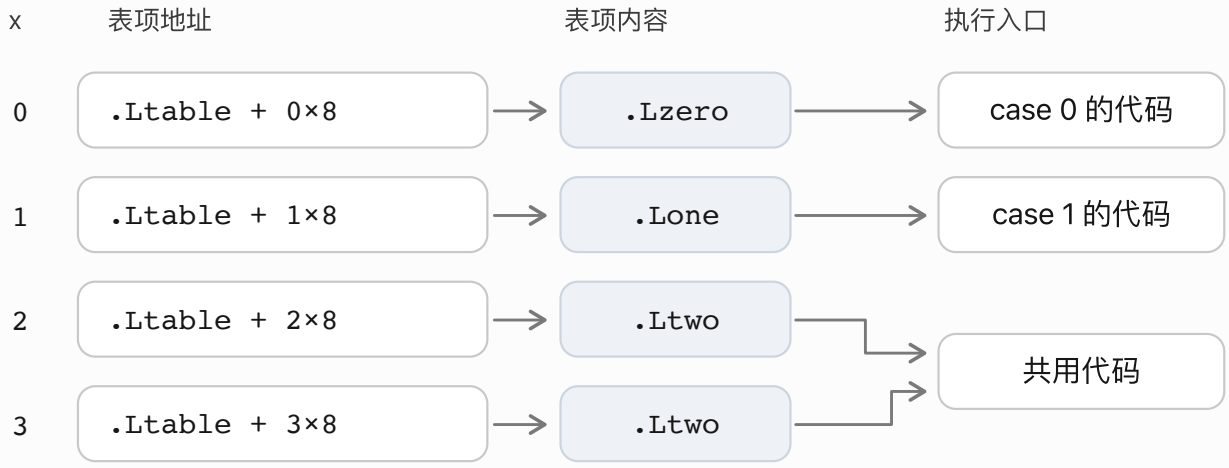
C 的 `switch` 根据一个整数值选择起点。`case` 是入口标签，`break` 才负责退出。两个相邻 `case` 可以共用同一段代码；某个 `case` 后没有 `break`，就会继续执行下一段，这称为 **fall-through，落入**。

不能看到 `case 0` 就只看它下面第一条语句。正确方法是：先用值找到入口，再沿实际控制流走到 `break`、`return` 或其他跳转为止。同理，汇编中两个不同表项可以填同一个标签；这是共用代码，不能因此认定表坏了。

2.1 跳转表里面存的是去哪里，不是计算结果

连续、比较密集的分支值适合用表定位入口。以下是本讲示例使用的绝对地址跳转表：每项用 `.quad` 存一个 8 字节代码地址。

跳转表：先读地址，再跳转



先定位表项地址，再读取其中保存的代码地址；索引 2 和 3 共用同一个入口。

读图要分三层：先计算表项的地址，再从该地址取出代码地址，最后跳到该代码地址执行。表项地址不等于表项内容；它也不是 `switch` 最终返回的数。这里的乘数 `8` 来自表项是 8 字节地址，和被操作的 C 数据是 `int` 还是 `long` 没有直接关系。

```
jmp *.Ltable(,%rdi,8)
```

这条指令用 `%rdi` 作索引，到 `.Ltable+8*%rdi` 处读一个目标地址，再跳过去。前面的 `*` 表示间接跳转；`jmp .Ltwo` 则是直接跳到给定标签。还有 `jmp *%rax`，目标地址直接来自寄存器值。不要把 `jmp *%rax` 和“去 `%rax` 所指的内存里再取一次目标”混为一谈。

先看case为0～3的表。 只有x=0、1、2、3才能直接用作这张表的索引；x<0或x>3时应去default。这里是“或”，不是两个条件同时成立。`cmpq $3,%rdi; ja .Ldefault` 按无符号解释检查x是否大于3：x=2时不跳，继续查表；x=4时跳到default；x=−1的64位位模式按无符号读为$2^{64}-1$，也跳到default。因此一次无符号上界检查就能排除范围两端。default是未匹配case时执行的代码，不代表固定返回某个数。

再换一个例子：case为10～13。 如果只放四个表项，表的索引仍为0、1、2、3，就要先算 `索引=x-10`：

原始x，即case值	索引x−10	查表去向
10	0	表[0]→case 10
11	1	表[1]→case 11
12	2	表[2]→case 12
13	3	表[3]→case 13

此时按无符号检查的是机器计算出的**索引**是否大于3，而不是原始x是否大于3。x=9得到低64位全1的索引，按无符号看很大，去default；x=14得到索引4，也去default；x=12得到索引2，进入case 12。减10只是调整查表编号，可以另用寄存器保存索引。读汇编恢复原case时要反推 `case值=索引+10`，所以表[2]对应case 12，不是case 2；这不要求程序额外执行一次加10。Q022：什么时候走default，为什么索引要减10、还原case又要加10？ 分别解释两个范围检查与原值/索引的区别。

有些机器代码用 4 字节相对偏移作表项，再加上基址得到目标，而不使用本节的 8 字节绝对地址表。遇到题目先看给定的表项格式；“跳表每项一定 8 字节”不是普遍规则。

示例 D3-E02：有落入和共享入口的 switch（往年题改编）

题源：2022 年期中无答案版（来源资料未随仓库分发），第三题，PDF 第 9—10 页。本例借用其“跳转表、落入、算术分支”的组织方式，减少分支、改小常数，去掉结构体写入，并明确所有路径的返回值。以下是改编后的完整题面与解答，不是原题原文。

参数是 `long x,long b,long c`，分别在 `%rdi,%rsi,%rdx`；取 `-10<=x<=10`、`-100<=b,c<=100`。

```
pick_value:
    cmpq $3, %rdi
    ja .Lpick_default
    jmp *.Lpick_table(,%rdi,8)
.Lpick_zero:
    subq $4, %rdx
.Lpick_one:
    leaq (%rsi,%rdx), %rax
    leaq (%rax,%rax,2), %rax
    ret
.Lpick_two:
    leaq 10(%rdx), %rax
    ret
.Lpick_default:
    movq %rsi, %rax
    ret

.section .rodata
.align 8
.Lpick_table:
    .quad .Lpick_zero
    .quad .Lpick_one
    .quad .Lpick_two
    .quad .Lpick_two
```

先列映射：0 去 `.Lpick_zero`，1 去 `.Lpick_one`，2 和 3 都去 `.Lpick_two`，范围外去默认分支。再看从每个入口出发执行了什么。

从 0 进入时先把 `c` 减 4，随后没有任何跳走指令，所以继续执行 `.Lpick_one`。两条 `lea` 先算 `b+c`，再算三倍，最后返回。从 1 进入时直接算三倍，**不会**先减 4。2 和 3 共用 `c+10`；默认返回 `b`。由此得到：

```
long pick_value(long x, long b, long c) {
    switch (x) {
    case 0:
        c -= 4;
        /* 有意落入 case 1 */
    case 1:
        return 3 * (b + c);
    case 2:
    case 3:
        return c + 10;
    default:
        return b;
    }
}
```

用 b=2,c=7 检查：x=0 返回 15，x=1 返回 27，x=2 和 x=3 都返回 17，x=-1 返回 2。共享入口只保证 2、3 执行同一段；0、1 虽然最后执行同一段，但 0 先改了 c，所以结果不同。

2.2 看见 cqto 和 idivq，不要继续把 rdx 当原参数

课程 switch 示例中有有符号除法。idivq divisor **执行前**读取一个 128 位被除数，它由 %rdx:%rax 拼成：%rdx 是高 64 位，%rax 是低 64 位。**执行后**商写入 %rax、余数写入 %rdx，覆盖它们原来的内容。高低位说的是输入，商和余数说的是输出；并非同一时刻保存两套值。冒号表示拼接位串，不是除号。

> Q023：rax、rdx为什么既是被除数的高低位，又是商和余数？用17÷5逐步看执行前后

若要把普通有符号 long a 除以 long b，通常先把 a 放 %rax，再用 cqto 把 %rax 的符号扩展到高半部分 %rdx。cqto 本身不做除法，也不改变 %rax；它会覆盖 %rdx，正数时补零，负数时补全一。

```
# 计算 a / b：a 在 %rdi，b 在 %rsi
movq %rdi, %rax
cqto
idivq %rsi
ret
```

例如 a=-13,b=5，商是 -2、余数是 -3，满足 -13=(-2)*5+(-3)。商向零截断，所以不能直接把它当算术右移的向下取整。除数不能为零；有符号最小值除以 -1 的商也无法表示。读题时遵守这些前提。

若除数原来恰好在 %rdx，必须在 cqto 前移到其他寄存器，否则会被覆盖。例如第三个参数 b 在 %rdx 时，可先 movq %rdx,%rcx，之后用 idivq %rcx。这是“寄存器的含义会随指令改变”的一个典型例子：不能从参数列表一直给 %rdx 贴着 b 的标签。

2.3 PC 相对跳转，只记一条计算规则

某些直接跳转在机器码中不保存完整目标地址，而保存一个有符号位移。计算规则是：**目标地址 = 该跳转指令结束后下一条指令的地址 + 位移**。例如跳转指令位于 0x400100，长度为 2 字节，位移为 -6，目标就是 0x400102-6=0x4000fc。

不能用当前指令的起始地址直接加位移，也不能默认所有跳转都只有两个字节；题目给了机器码长度就按长度算。负位移常见于循环回边。PC 相对跳转的“相对”说的是目标地址的编码方式，和“是否有条件”“是否是函数调用”不是同一个分类维度。

3. 调用：把“去哪里”和“带什么”分开

y=f(x) 看起来只是一句 C，机器至少要解决三件事：把控制流交给 f，把参数给 f，并保证 f 完成后能回到这句调用之后。函数若还需要局部数据，就要考虑它们放在哪里、何时不再使用。

返回地址回答“执行完后回到哪里”，它是代码地址。**返回值**回答“函数算出了什么”，例如 f(3) 得到的 9。本讲整数返回值通常在 %rax；返回地址由 call 放入栈中。返回地址既不是“函数入口地址”，也不是“调用指令本身的地址”。

调用者与被调用者是相对身份：如果 A 调 B、B 又调 C，B 对 A 是被调用者，对 C 是调用者。寄存器保存题一定要先说清正在看哪一次调用，否则会把责任颠倒。

3.1 栈是一片内存，rsp 是地址，不是数据盒子

把栈理解成按后进先出使用的一段内存。x86-64 中，栈向较低地址增长，%rsp 保存当前栈顶的地址。**低地址增长**意味着新压入一个 8 字节对象时，%rsp 减 8；不是因为数值变小就“释放了空间”。

以下描述针对常见的寄存器操作数；本讲不讨论以 %rsp 自身为 pop 目的等特殊情形。

指令	常见操作的含义
`pushq %rbx`	先把 `%rsp` 减 8，再把 `%rbx` 中保存的值复制到新栈顶的 8 字节内存；`%rbx` 不变
`popq %rbx`	从 `%rsp` 指向的内存读 8 字节，写入 `%rbx`，再把 `%rsp` 加 8
`call f`	将紧接在本条 call 后面的指令地址压栈，再开始执行函数 `f` 入口处的指令
`ret`	从当前栈顶读出返回地址，`%rsp` 加 8，再从读出的代码地址继续执行

`pushq %rbx` 保存的是寄存器里的值，不是“寄存器自身的地址”，也不会去读取 `%rbx` 所指向的内存。若 `%rbx=42`，压入的就是 42；若它保存一个指针，压入的才是那个指针值。`popq %rbx` 取回的是栈内存里的内容，而不是栈顶地址。`ret` 返回的是执行位置，普通整数返回值通常由 `%rax` 提供；若函数使用过额外栈空间，须在 `ret` 前让栈顶重新指向返回地址。

> Q024：push/pop保存的是值还是地址？call转到f、ret返回分别在做什么？

`pop`、`ret` 使栈顶越过一块内存，不代表那块内存立刻被清零；它只是退出当前活跃栈区。`subq $16,%rsp` 也只是预留 16 字节，不负责把它们初始化。考试中若内存从未写过，就不能擅自给它填零。

3.2 一次 call 的三个时刻

假设调用指令开始前 `%rsp=0x1000`，调用后的下一条指令地址记作 `R`。被调用函数先不做任何额外压栈操作。

一次 call 与 ret 的三个时刻

A. 调用前

`0x1000`
原有栈数据 ← rsp

`0x0ff8`
本次调用尚未使用

此时还没有执行 call；rsp 指向 0x1000，返回地址 R 尚未压入本次调用的栈区。

B. 刚进入被调用函数

`0x1000`
原有栈数据

`0x0ff8`
R ← rsp

call 先将 rsp 减 8，再在 0x0ff8 写入返回地址 R；控制流转到被调用函数。

C. 执行 ret 之后

`0x1000`
原有栈数据 ← rsp

`0x0ff8`
旧字节可能还在

ret 从 0x0ff8 取出 R 并使 rsp 加 8；控制回到 R。弹出不意味着旧内存被清零。

每栏中地址由高到低排列。粗边框指出 rsp 当前指向的位置；弹栈不等于清空旧字节。

按图追踪两个变化：`call` 让 `%rsp` 从 `0x1000` 变为 `0x0ff8`，且写入 `R`；`ret` 从 `0x0ff8` 取出 `R`，让 `%rsp` 恢复到 `0x1000`。被调用函数若在中间用了额外空间，必须在 `ret` 前把栈顶恢复到存放 `R` 的位置，否则 `ret` 会把别的数据误当成地址。

4. ABI：函数之间约定怎么交接

ABI 是二进制层面的约定。在本讲使用的 System V AMD64 ABI 下，普通整数或指针类参数的前六个位置如下。这里只处理简单标量参数；浮点和部分结构体有自己的分类规则，下一讲再讲。

参数次序	1	2	3	4	5	6
寄存器	`%rdi`	`%rsi`	`%rdx`	`%rcx`	`%r8`	`%r9`

第七个及后面的普通 `long` 参数通过栈传递。**刚进入被调用函数、尚未改变 `%rsp` 时**，`0(%rsp)` 是返回地址，第七个参数位于 `8(%rsp)`，第八个位于 `16(%rsp)`。如果函数已经 `push` 或 `sub` 过 `%rsp`，偏移必须相应重算，不能永远写 `8(%rsp)`。

可以用从右往左 `push` 的方式构造栈参数：先第八个，再第七个，这样第七个离返回地址最近。编译器也能一次预留空间后用 `mov` 填入，因此“参数一定由 push 指令传递”同样不成立。调用者负责回收它自己为这些栈参数准备的空间。

4.1 为什么有的值跨一次 call 就丢了

寄存器数量有限，被调用函数要用它们工作。假设A调用B，A是调用者、B是被调用者；它们使用同一组寄存器，进入B时不会自动换一套。ABI按“谁负责保留旧值”将寄存器分成两类：

- **调用者保存**：`%rax,%rcx,%rdx,%rsi,%rdi,%r8,%r9,%r10,%r11`。B可以改它们，无须恢复；如果A在B返回后还需要其中旧值，由A在调用前保存到安全位置。
- **被调用者保存**：`%rbx,%rbp,%r12,%r13,%r14,%r15`。函数可以使用它们，但若改了值，就要在返回前恢复成进入时的值。`%rsp` 也必须恢复到满足正常返回的位置，但通常不把它当一般临时寄存器列在这组里。

“调用者保存”不等于调用前必须无条件保存全部寄存器；只保存后面还需要的值。“被调用者保存”也不等于每次进入函数必须压入全部六个；没修改就不需要保存。

例如A的 `%rdi=10`，B把它改成99后可以直接返回，所以A若还需要10，要提前另存。若A的 `%rbx=10`，B想把它改成99，B就要先保存10，并在返回前恢复为10；A可以依赖这个约定。保存和恢复是程序指令完成的，不是call/ret自动备份整组寄存器。

> Q025：调用者保存、被调用者保存，究竟是谁保存、什么时候需要保存?

假设你在 `%rdi` 保存 `x`，调用 `f` 后还要用 `x`。即使 `f` 恰巧在某一次运行中没改 `%rdi`，也不能仅凭 ABI 保证它不变。稳妥做法是把 `x` 存到当前函数的栈空间，或放入自己已经妥善保存的 `%rbx`。后一种选择意味着当前函数也要对它的调用者负责，在最后恢复原 `%rbx`。

4.2 栈帧与 rbp：常见布局不是强制模板

一次尚未返回的函数调用，可以有属于它的保存信息、局部数据和临时空间。把这部分组织称作栈帧，有助于追踪递归。但编译器可能省略不需要的空间，直接用寄存器计算；也可能省略帧指针。因此不能背成“每个函数一定 `push %rbp`，每个局部变量都在栈上”。

先把栈帧理解成一次函数调用在栈上使用的区域。`%rbp`本身是寄存器；采用帧指针时，它保存这块区域里的一个固定基准地址，方便用偏移定位数据。`%rsp`保存当前栈顶地址，会随着栈空间的使用而变化。

> Q026：栈帧和rbp是什么？用具体地址逐步拆解函数开头与leave/ret

一种常见开头是：

```
pushq %rbp
movq %rsp, %rbp
subq $16, %rsp
```

执行后 `%rbp` 固定指向保存的旧 `%rbp`。在这种具体布局里，`0(%rbp)` 是旧 `%rbp`，`8(%rbp)` 是返回地址；局部数据通常在负偏移处。`%rsp` 会随着临时压栈而变，`%rbp` 则提供一个相对稳定的基准。

对应结尾可以写 `movq %rbp,%rsp; popq %rbp; ret`，其中前两条的效果可由 `leave` 完成。`leave` **不负责跳回调用者**，仍然需要 `ret`。本讲练习会明确使用哪种栈帧，不能将这张布局强套到没有帧指针的函数上。

如果代码需要把局部变量的地址交给一个实际执行内存访问的外部函数，就必须提供可寻址的内存位置，栈是常见选择。但若编译器能把整个过程内联、消除，源代码中写了 `&x` 也不保证最终机器码一定保留栈槽。考试给出汇编时，以实际指令为准；分析一个普通局部整数时，先看它是否始终只放在寄存器里。

4.3 栈对齐：看清是在 call 前还是入口处

对于本讲只用普通整数、指针的调用情形，**执行 `call` 之前，`%rsp` 应为 16 的倍数**。`call` 再压入 8 字节返回地址，所以**刚进入被调用函数时，`%rsp` 对 16 取模为 8**。两句话并不矛盾，它们描述不同时间点。更强对齐要求的特殊参数不在本讲练习范围内。

例：入口 `%rsp=0x1008`；`pushq %rbx` 后变成 `0x1000`，这时已按 16 对齐；再 `subq $16,%rsp` 后是 `0x0ff0`，仍然对齐，可以调用其他函数。如果入口直接减 16，就得到 `0x0ff8`，仍余 8，不能仅因为“分配的字节数是 16”就认为调用点对齐了。

对齐不是多画一个框就完成的。解题时记录每条 `push/pop/sub/add/call/ret` 的增减量，并在每个 `call` 前检查一次模 16。为了满足对齐而预留的空位不是自动出现的新 C 变量，也不能因为某个栈槽存在就推断它有特定内容。

示例 D3–E03：局部变量取地址，调用后再使用（助手补充）

先看两个 C 函数，输入 `-100<=x<=100`：

```c
long bump(long *p, long delta) {
    long old = *p;
    *p = old + delta;
    return old;
}

long use_bump(long x) {
    long v = 20;
    long old = bump(&v, 3);
    return x + v + old;
}
```

`bump` 返回修改前的值，同时通过指针修改 `v`。因此必须分清“返回的 `old`”和“写回内存的新值”。以下是一个符合本讲约定的实现：

```asm
bump:                       # 函数入口：rdi存指针p，rsi存增量delta
    movq (%rdi), %rax        # 读取*p的旧值到rax，留作返回值old
    addq %rsi, (%rdi)        # 将delta加到*p上，更新内存中的v
    ret                     # 返回use_bump，rax中仍是旧值old

use_bump:                   # 函数入口：rdi存输入x
    pushq %rbx              # 保存外层调用者的rbx，返回前要恢复
    subq $16, %rsp          # 预留16字节栈空间，并保持调用前16字节对齐
    movq %rdi, %rbx         # 将x存入rbx，让它在调用bump后仍可用
    movq $20, 8(%rsp)       # 在栈内存中初始化局部变量v=20
    leaq 8(%rsp), %rdi      # 将v的地址作为第1个参数p，不是读取v的值
    movl $3, %esi           # 设置第2个参数delta=3；写esi也将rsi高32位清零
    call bump              # 调用bump(&v,3)；返回后rax=20，内存中v=23
    addq 8(%rsp), %rax      # 将修改后的v加到old上：rax=20+23=43
    addq %rbx, %rax         # 再加输入x，得到返回值x+v+old=x+43
    addq $16, %rsp          # 回收预留的16字节，让rsp指向保存的旧rbx
    popq %rbx               # 恢复外层的rbx，并让rsp指向外层返回地址
    ret                     # 返回外层调用者，rax中保留计算结果x+43
```

先问每个位置装什么。 `%rbx` 保存输入 `x`，供调用后使用；`8(%rsp)` 是 `v` 的内存位置；`lea` 把这个位置的地址给第一个参数 `%rdi`，没有读取 `v`；第二个参数 `%rsi` 是 3。若错写 `movq 8(%rsp),%rdi`，传过去的是数值 20，被当指针使用就完全不是原程序了。

再追一次栈。 假设进入 `use_bump` 时 `%rsp=0x1008`，此时 `0x1008` 已放着它返回到外层的地址 `R外`。



图中地址由高到低排列。`R内` 是 `call bump` 后第一条 `addq` 的地址。`bump` 入口的 `%rsp` 改变了，但 `%rdi` 内保存的地址仍是 `0x0ff8`，不会自动跟着 `%rsp` 变。它将该处从 20 改成 23，并把旧值 20 放在 `%rax`。

`bump` 的 `ret` 弹出 `R内`，返回 `use_bump` 后 `%rsp` 又是 `0x0ff0`。两条加法得到 `20+23+x`，所以 `x=5` 时返回 48。随后释放 16 字节、恢复原 `%rbx`，`%rsp` 回到 `0x1008`，最后 `ret` 才弹出 `R外`。

这里没有使用 `%rbp`，照样能管理栈。保存 `%rbx` 是为了对外层调用者履行义务；把 `x` 放进去是为了让 `bump` 按约定保住它。两个理由对应不同层次，必须同时说清。

5. 递归：重复的是代码，分开保存的是每次调用的状态

递归就是函数调用自己。没有另一套特殊 `recursive call` 指令，仍使用 `call` 和 `ret`。难点在于同一段代码同时有多次调用尚未结束，每次都需要知道“我自己的参数是什么、子调用返回后接着做什么”。

先把递归拆成两阶段。**向下调用**时，当前层暂时停住，保存稍后要用的值，进入下一层；**向上返回**时，子层把结果放到 `%rax`，当前层取回自己保存的值，完成剩下的计算，再返回。函数名一样不代表局部状态共用一个槽。

示例 D3-E04：每一层都要保留自己的 n（助手补充）

```c
/* 0 <= n <= 100 */
long sum_to(long n) {
    if (n == 0) return 0;
    return n + sum_to(n - 1);
}
```

一种实现如下：

```asm
sum_to:
    testq %rdi, %rdi
    je .Lsum_base
    pushq %rbx
    movq %rdi, %rbx
    subq $1, %rdi
    call sum_to
    addq %rbx, %rax
    popq %rbx
    ret
.Lsum_base:
    movl $0, %eax
    ret
```

入口先检查 `n==0`。基本情形直接返回零，没有额外保存 `%rbx`；因此基本情形路径也不能执行 `popq %rbx`，否则会把返回地址错当保存值弹掉。非基本情形先保存进入本层时的旧 `%rbx`，再把本层 `n` 放到 `%rbx`；把参数改为 `n-1` 后递归调用。

为什么不直接等调用回来再加 `%rdi`？它是调用者保存寄存器，而且我们已经把它减一传给子层，不能再用它当本层原来的 `n`。为什么子层也使用 `%rbx` 却不会破坏本层？因为子层同样先保存进入时的 `%rbx`，返回前恢复。这条约定层层成立，便形成各层独立状态。

对 `sum_to(3)`，把调用和返回分开写：



读图时先从左上向右下走，直到基本情形；再从右下往上代入结果。最大同时活跃调用数为 4，调用自己形成的递归边数为 3。这两个数不同。虽然基本情形没有额外 `push %rbx`，它依然有由调用它的 `call` 压入的返回地址。

每个非基本层入口的 `%rsp` 对 16 余 8，执行一次 `push %rbx` 后余 0，恰好满足再调用自己的对齐要求。返回时 `pop` 与本层的 `push` 配对。这里不能随意把 `push` 删掉再说“只用一个寄存器就够了”，它同时参与保存旧值与调用点对齐。

5.1 双递归还多一个问题：第一个结果也要保住

考虑 `f(n)=f(n-1)+f(n-2)`。第一次 `call` 的结果在 `%rax`；第二次 `call` 也会把自己的结果放进 `%rax`。如果不保存第一个结果，最后只有第二个结果了。因此，两次调用之间必须把第一次结果转移到栈槽或妥善使用的被调用者保存寄存器。

这种问题可见于 2023 年期中带答案版（来源资料未随仓库分发）第三题，PDF 第 16—17 页的组合数递归代码。本讲只指出其训练价值，不要求今天把整道长题全部补完。遇到多次 `call`，给每个跨越 `call` 仍然活跃的值画一条“必须保存”的标记，比盲目给每行写中文更有效。

还要区分“总调用次数”与“最大栈深”。两个递归调用通常先后完成，不是同时在栈上铺开一整棵树。最大栈深看一条尚未返回的调用链；总调用次数才统计整棵调用树。今天先练单链递归，确保每层状态和返回顺序能独立写对。

6. 遇到长题时，按这个顺序落笔

循环题先给变化寄存器命名，找到初始化与回边，再翻译比较的方向，最后用零、一次循环、恰好到边界三个输入核对。若反推的 C 和原作者写法不同，只要行为和题目要求一致，不必为了猜原代码而把正确解改错。

switch 题先记参数寄存器，再看索引是否平移和是否检查范围，逐项列出“索引→标签”，最后从标签沿代码执行到终点。先完成映射表再算结果，能避免漏掉共享入口和落入。出现 `idiv` 时，单独标出 `%rax/%rdx` 被重新赋予的含义。

调用题先画当前 `%rsp` 和返回地址，再逐条记栈顶变化；参数和返回值另写在寄存器旁边。对于每次 `call`，只问三件事：参数现在对不对？调用后还需要的值保存了吗？此刻栈是否对齐？`ret` 前再核对栈顶是否指向正确返回地址。

递归题先做最小输入。若基本情形还不清楚，直接追大输入只会把错误放大。然后画两层，明确每层保存什么、子结果放在哪里，最后才推广到更多层。不要用“递归会自动保存所有寄存器”作解释；只有给定指令和 ABI 约定承担这些工作。

7. 独立练习：4 道必做，共 40 分

以下均为助手补充的新题。先只看题面，写出推理；每题 10 分，四题合计 40 分，基础过关线为 32 分。除总分外，必须至少独立画对一次调用前、入口、返回后的栈变化。得到提示后完成的部分可以订正，但不计入首次独立正确率。每题的分值在小问后给出，评分过程另见答案文件。

D3-P01：恢复循环并核对边界（10 分）

`long loop3(long n)` 的输入满足 `0<=n<=100`，`n` 在 `%rdi`：

```
loop3:
    movl $0, %eax
    movl $1, %edx
    jmp .Lp1_test
.Lp1_body:
    leaq (%rdx,%rdx,2), %rcx
    addq %rcx, %rax
    addq $2, %rdx
.Lp1_test:
    cmpq %rdi, %rdx
    jle .Lp1_body
    ret
```

1. 说明 `%rax` 和 `%rdx` 分别承担什么角色。（1 分）
2. 写出初始化、继续循环的条件、每轮更新。（2 分）
3. 写出等价且完整的 C 函数，不能直接用闭式求和公式替代循环。（2 分）
4. 对 `n=5`，列出每轮加入的数及最终返回值。（1 分）
5. 对 `n=0`，是否执行循环体？说明依据。（1 分）
6. 只把 `jle` 改成 `jl`，`n=5` 时返回值如何变化？（1 分）
7. 写出一句在每次到达 `.Lp1_test` 时成立的关系，说明当前累计值与 `%rdx` 的关系，并解释为什么更新后仍成立。（2 分）

D3-P02：跳转表、默认分支与落入（10 分）

参数 `long x,long a` 在 `%rdi,%rsi`，输入 `-10<=x<=10`、`-100<=a<=100`。给定：

```
route:
    cmpq $4, %rdi
    ja .Lp2_default
    jmp *.Lp2_table(,%rdi,8)
.Lp2_a:
    addq $2, %rsi
.Lp2_b:
    leaq (%rsi,%rsi,4), %rax
    ret
.Lp2_c:
    leaq -3(%rsi), %rax
    ret
.Lp2_default:
    movq $-1, %rax
    ret

.section .rodata
.align 8
.Lp2_table:
    .quad .Lp2_a
    .quad .Lp2_b
    .quad .Lp2_default
    .quad .Lp2_c
    .quad .Lp2_c
```

1. 写出 x=0,1,2,3,4 分别进入的标签。(2 分)
2. 为何 x=-1 也会去默认分支？说明 ja 的比较方式。(1 分)
3. 哪些 x 共用入口？哪一处存在落入？ (1 分)
4. 写出完整等价 C 函数，显式表示 case 与默认分支。(2 分)
5. 分别计算 route(0,4)、route(1,4)、route(3,4)。(1 分)
6. 假设表基址是 0x5000，x=4 时读哪个表项地址？该处存的是结果还是代码地址？ (1 分)
7. 若将间接跳转换成 leaq .Lp2_table(,%rdi,8),%rax; jmp *%rax，还能正确执行吗？说明错在“取表项地址”和“读表项内容”的哪一步。(2 分)

D3-P03：局部内存、调用前后与栈恢复（10 分）

以下 caller 输入 -100<=x<=100，x 在 %rdi。进入 caller 时 %rsp=0x2008，内存 [0x2008] 为返回到外层的地址 R0，原 %rbx=99。把 call read_add 后的第一条指令地址记为 R1。未明确写入的栈内存内容未知。

```
caller:
    pushq %rbx
    subq $16, %rsp
    movq %rdi, %rbx
    movq $7, 8(%rsp)
    leaq 8(%rsp), %rdi
    movl $5, %esi
    call read_add
    addq %rbx, %rax
    addq $16, %rsp
    popq %rbx
    ret

read_add:
    movq (%rdi), %rax
    addq %rsi, %rax
    ret
```

1. 执行 pushq %rbx、subq $16,%rsp 后，分别给出 %rsp；保存的 99 在哪个地址？（2 分）
2. 紧挨 call 之前，%rdi 和 %rsi 的值分别是什么？该处是否满足 16 字节对齐？（1 分）
3. 刚进入 read_add 时，%rsp 是多少？R1 在哪个地址？画出 R0、旧 %rbx、局部值 7 与 R1 的相对位置。（2 分）
4. 对 x=4，read_add 返回值与 caller 最终返回值分别是多少？（1 分）
5. read_add 返回到 caller 后，及 caller 最终返回外层后，%rsp 分别是多少？最终 %rbx 为多少？（2 分）
6. 如果删去保存 x 到 %rbx 的做法，并将调用后的加法改成 addq %rdi,%rax，能否得到 x+12？指出本段具体代码里 %rdi 已经变成什么，并说明一般情况下的 ABI 风险。(2 分)

D3-P04：递归每层保存什么（10 分）

odd_sum(long n) 的输入满足 0<=n<=20。给定：

```
odd_sum:
    testq %rdi, %rdi
    je .Lp4_base
    pushq %rbx
    leaq -1(%rdi,%rdi), %rbx
    subq $1, %rdi
    call odd_sum
    addq %rbx, %rax
    popq %rbx
    ret
.Lp4_base:
    movl $0, %eax
    ret
```

1. 写出基本情形与递推关系。(2 分)
2. 写出完整等价 C 函数。(1 分)
3. 计算 odd_sum(3)，分别列出向下各非基本层保留的值和向上各层返回的值。(2 分)
4. 为什么递归调用后还能用 %rbx？“被调用者保存”在父层和子层各起什么作用？（1 分）
5. 计算 odd_sum(3) 时，最多同时活跃几个本函数调用？其中有几个非基本层的 push %rbx 尚未匹配 pop？（1 分）
6. 假设进入最外层 odd_sum(3) 时 %rsp=0x3008，到 odd_sum(0) 刚入口时 %rsp 是多少？说明每下降一层的变化，并检查非基本层递归调用前的对齐。(2 分)
7. 基本情形路径为什么不能照搬非基本路径执行一次 popq %rbx 再 ret？（1 分）

D3-P05：超过六个参数的调用（选做，10 分）

已完成四道必做题并能正确画栈后再做。本题不计入前面的 40 分。假设紧挨第一条 push 前 %rsp=0x4000，前六个 long 参数 1—6 已正确放入六个参数寄存器：

```
    pushq $8
    pushq $7
    call eight
    addq $16, %rsp
    # 回到调用者的后续代码

eight:
    movq 8(%rsp), %rax
    addq 16(%rsp), %rax
    addq %rdi, %rax
    ret
```

1. 列出前六个参数的寄存器。(1 分)
2. 两次压栈后、刚进入 `eight` 时的 `%rsp` 分别是多少？ (2 分)
3. 在 `eight` 入口画出返回地址、第七、第八参数的位置。(2 分)
4. `eight` 返回什么值？它修改了哪些寄存器？是否破坏任何被调用者保存寄存器？ (2 分)
5. 为什么需要调用后的 `addq $16,%rsp`？执行后栈顶是多少？ (1 分)
6. 如果交换两次 `push` 的顺序，参数布局会怎样变化？为什么仅凭这次返回值未改变，也不能说交换是正确的？ (2 分)

8. 本讲完成时应留下什么

留下四道题的首次答案、至少一张调用栈图，以及实际出现的错误。若循环条件错了，记录具体是哪条 `cmp` 被读反；若栈错了，记录是哪一次 8 字节变化遗漏；若递归错了，记录哪个跨调用的值没有保存。不要只写“汇编不熟”或“递归不会”，这样的标签无法指导第二次练习。

分数达到 32 分后，再检查三个必须能口述的结论：为什么跳表要先读表项再跳转；返回地址和返回值分别在哪里；为什么父子递归层都使用 `%rbx` 仍能各自保留状态。总分够而这三点不清楚时，先回到对应示例重画一次，不追加更长递归。

来源与范围核对

- 课程第 5 讲：Control（来源资料未随仓库分发）：PDF 第 37—47 页为循环，第 49—58 页为 switch、跳转表及除法，第 62—64 页为反汇编中的跳转表。
- 课程第 6 讲：Procedures（来源资料未随仓库分发）：PDF 第 10—24 页为栈和控制传递，第 26—27 页为参数，第 29—53 页为栈帧与局部存储，第 54—65 页为保存约定，第 67—75 页为递归及总结。
- 第 5 讲普通研讨题（来源资料未随仓库分发）：研讨 2(3)(5)—(9)；第 6 讲普通研讨题（来源资料未随仓库分发）：研讨 1(4)(5)(7)、研讨 2(1)—(4)(6)—(8)。这里只借用课内概念要求，不安排外部调研或拓展任务。
- 教材对应 3.4.4（压栈与弹栈）、3.5.5（特殊算术操作）、3.6.3—3.6.4（跳转指令与编码）、3.6.7—3.6.8（循环和 switch）、3.7（过程）。教材 PDF（来源资料未随仓库分发）仅按小节定位，不猜填印刷页码。

除标明的往年题改编外，本讲示例、练习、图示与讲解均为助手补充；涉及的机器规则以所述课程环境为前提。往年题带答案材料只作核对来源，不作为未经检查的正确性保证。

第4讲 数据布局与内存安全

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

1 先把问题变成地址计算
2 结构体要满足成员与整体的对齐
3 联合是共享同一片字节
4 浮点数在汇编里走另一组寄存器
5 程序内存布局与越界
6 ROP 只需先看懂 ret 链
7 独立练习
8 收尾与来源

这是复习计划的第4天，对应课程第7、8讲，并复用前面学过的寻址和调用栈。读完后，你应能把一个 C 对象画成字节，算出成员或元素的位置，并解释越界写入为什么会影响程序控制流。

本讲采用课程的 x86-64、Linux、System V ABI：char 1 字节，short 2，int/float 4，long/double/普通指针 8；默认小端和常见自然对齐，不使用 packed。下文的大小和偏移依赖这些前提，不是所有 C 平台的固定规则。

1 先把问题变成地址计算

前一讲中，`movq (%rdi), %rax` 的意思是：把 rdi 里的数当作地址，从那里读取八字节。现在遇到 `a[i]`、`p->value`，机器仍然做这件事；区别只是先算出应该访问哪个地址。

读本讲前，用自己的话回答：`%rdi`、`(%rdi)`、`8(%rdi)` 分别是什么？前者是寄存器内容，后两者分别访问地址 rdi 和 rdi+8 处的内存。若把这三者混淆，先回第1讲的寻址部分。

一维数组就是等宽格子

```
int a[4] = {11, 22, 33, 44};
```

如果 a 的首地址是 0x1000，每个 int 占四字节：

图4-1 数组下标与字节地址。 从左到右地址增加；一个方框代表一个 int，而不是一个字节。



所以 `&a[i] = 首地址 + i × sizeof(int)`。a+1 跨过一个 int，增加四字节，不是增加一个字节；先转换成 `char *` 再加一，才跨过一个字节。

表达式	含义	是否需要读取数组元素
a+i、&a[i]	第 i 个元素的地址	否
a[i]、*(a+i)	第 i 个元素的值	是
sizeof(a)	整个数组的字节数，此处 16	否
sizeof(a[0])	一个元素的字节数，此处 4	否

在大多数表达式里，数组名会转换为指向首元素的指针；**数组本身不是指针变量**。sizeof(a) 和 &a 是理解这一点最有用的例外。函数形参写 int a[4] 时会调整为指针参数，因此不能在函数内用 sizeof(a) 求传入数组的总大小。

二维数组先跨行 再跨列

```
int b[2][3] = {{10, 11, 12}, {20, 21, 22}};
```

图4-2 二维形状与实际连续存储。 图的上半部分方便理解下标，下半部分才是内存的排列顺序。



如果每行 C 个元素，每个元素 K 字节：

&b[i][j] = B + K × (i × C + j)。

先乘列数 C，算出前 i 行共有多少元素；加 j 找到本行第 j 个；最后乘 K 变成字节偏移。**行号乘的是列数，不是总行数。**

例 D4-E1 从 C 下标到机器地址

助手补充例。long a[3][5] 首地址是 0x2000，求 a[1][3] 的地址，并解释 a+1。

1. 每个 long 八字节，每行五个，所以一行占 40 字节。

2. `[1][3]` 前面有 `1×5+3=8` 个元素，偏移 `8×8=64=0x40` 字节。
3. 地址是 `0x2040`。这个地址处的八字节是什么数，题面没有给出，不能凭地址猜值。
4. `a` 转换为“指向一行的指针”，`a+1` 增加 40 字节，结果是 0x2028。

若 i 在 rdi、j 在 rsi，计算元素下标 `5*i+j` 可写成：

```
leaq (%rdi,%rdi,4), %rax  # rax = 5*i，只做算术
addq %rsi, %rax           # rax = 5*i+j
# 再以8为比例访问 long 元素；首地址须另有寄存器或符号提供
```

当列数 n 在运行时才知道时，公式仍是 `i*n+j`，只是不能把 n 烘焙成固定常数。例如 `int get(size_t n, int a[][n], size_t i, size_t j)` 中，需要在运行时乘 n。这不是“教材中动态分配栈帧”的同一个问题。

别把指针数组当成二维数组

声明	从变量名向外读	+1 的步长
`int a[2][3]`	a 是两行，每行三个 int	表达式 `a+1` 跨一行，12 字节
`int *p[3]`	p 是三个指针组成的数组	`p+1` 跨一个指针，8 字节
`int (*q)[3]`	q 是指向三个 int 的数组的指针	`q+1` 跨一行，12 字节

`a[i][j]` 可以直接用行宽计算地址；`p[i][j]` 通常先读取指针 p[i]，再从它指向的地方访问第 j 个 int。p 的三行可以位于不相邻的区域，因此 `int **` 不能直接替代 `int (*)[3]`。

数组题先写类型，再算步长。看到 `sizeof` 时，先判断对象是数组还是指针，不能仅凭表达式长得像地址就都填八。

2 结构体要满足成员与整体的对齐

这一节要解决两个问题：结构体里的每个成员放在哪里，整个结构体占多少字节。成员大小之和不一定等于结构体大小，因为为了让成员从合适的地址开始，可能需要在中间和末尾留空。“对齐”约束的是存放地址，不是成员的数值。

> Q027：结构体对齐在讲什么？为什么成员合计15字节，结构体却占24字节？

结构体的字段按声明顺序放置，但中间可能留空。空位叫填充，不是隐藏变量，也不保证值为零。

常见自然对齐规则下：char 对齐 1，short 对齐 2，int/float 对齐 4，long/double/指针对齐 8。对齐为 K 的字段，其起始地址应是 K 的倍数；结构体整体大小是最大成员对齐要求的倍数，保证结构数组的下一个对象也能正确对齐。

计算时固定做四步：

1. 当前偏移从 0 开始。
2. 放每个成员前，把偏移向上调整到该成员对齐要求的倍数。
3. 记录成员起点，再加上成员大小。
4. 所有成员放完后，把总长度向上调整到结构体整体对齐要求的倍数。

向上对齐可以写为 `align(x,K)=ceil(x/K)*K`。K 是二的幂时也常见 `(x+K-1)&~(K-1)`；考试手算直接找下一个倍数即可。

例 D4–E2 真题中的结构体布局

原题截取并展开讲解：2025 期末无答案版（来源资料未随仓库分发），第一大题第4题，PDF 第3页。这里只讨论题中的结构体，不使用该卷其他范围的题。

```
struct S {
    char c;
    int i;
    short s;
    double d;
};
```

字段	大小	放置前偏移	对齐后起点	放置后偏移
c	1	0	0	1
i	4	1	4	8
s	2	8	8	10
d	8	10	16	24

整体对齐为 8，24 已是八的倍数，因此 `sizeof(struct S)=24`。

图4–3 结构体的字节布局。 点表示填充；数字是相对结构体起点的字节偏移。

结构体：填充让每个字段正确对齐

struct S · 总大小 24 字节 · 整体对齐 8



`c@0 → i@4 → s@8 → d@16 → 结束于24`

每个小格代表一个字节；带点的灰格是填充，不是额外字段，也不保证内容为零。

把顺序改为 `double d; int i; short s; char c;` 后，四个成员分别起于 0、8、12、14，实际成员结束于 15，再补一个字节，大小变成 16。声明顺序确实会影响空间；编译器不能擅自把原结构体字段重排成这种顺序。

从汇编反推字段时 先写约束

若 p 在 rdi，`movl 12(%rdi),%eax` 说明读取 p 起点后 12 字节处的四字节数据。它可能对应一个 int 字段，也可能是数组中的 int；只凭这一条指令不一定能确定完整类型。

例如：

```
struct R { char tag; int a[N]; long total; };
```

tag 起于 0，a 起于 4。若 total 起于 24，则 `align(4+4*N,8)=24`，所以 `16 < 4+4*N <= 24`，正整数 N 只能为 4 或 5。**不能凭“24−4”直接除以四得到唯一的 5，因为可能存在填充。** 要得到唯一答案，题目还需给其他约束。

3 联合是共享同一片字节

> Q028：联合这一节在讲什么？为什么写b的字节，也会改变w?

结构体的字段各占一段空间；联合的字段从同一起点开始，相互重叠。联合大小至少容纳最大的字段，并满足整体对齐，不是把各字段大小相加。

例 D4–E3 同一组字节按不同方式解释

助手补充例，按课程的 C 实现和整数表示前提分析：

```
union U {
    unsigned char b[4];
    unsigned int w;
};
```

写入 `b[0..3]={0x11,0x22,0x33,0x44}`。这四次写入改变的是同一个联合的四个字节。

图4–4 共享存储与字节序。 地址从左到右增加；改变解释方式不会重新搬动字节。



小端下最低地址放整数低位字节，因此 w 是 0x44332211；如果题目明确指定大端，w 是 0x11223344。这个联合大小是 4，若改为有这两个字段的结构体，大小则为 8。

位重解释与类型转换不同：把 float 的字节当作整数位模式读取，不等于执行 `(int)f`。后者试图把数值转换成整数。跨类型读写还涉及语言规则；本讲联合题按题给 C 实现处理，不把 `*(float *)&some_int` 当成普遍合法的 C 写法。查看对象字节可使用 `unsigned char *`，复制位模式可使用 `memcpy`。

4 浮点数在汇编里走另一组寄存器

整数与指针通常走通用寄存器，标量浮点参数走 XMM 寄存器。这是两套分别计数的参数通道，不是按所有参数的总顺序依次分配通用寄存器。

在本课程 Linux System V 的普通函数调用中，寄存器可容纳的前几个标量浮点参数依次用 xmm0—xmm7；返回 float 或 double 常用 xmm0。XMM 寄存器是调用者保存。更复杂的结构体参数分类和可变参数函数不在这里展开。

指令中的记号	本讲理解方式
`ss`	标量单精度，float
`sd`	标量双精度，double
`movss`、`movsd`	浮点数据传送；结合操作数判断读内存还是写内存
`addss`、`addsd`	相应精度的浮点加法
`cvtss2sd`	float 数值转换为 double
`cvtsi2sd`	有符号整数转换为 double，整数宽度由具体指令形式决定
`ucomiss`、`ucomisd`	标量浮点比较，设置 ZF、PF、CF

例 D4–E4 混合参数如何分配

助手补充例：

```
double combine(int n, double x, float y, double *out) {
    double r = x + (double)y + (double)n;
    *out = r;
    return r;
}
```

这里 n 使用 edi，x 使用 xmm0，y 使用 xmm1，out 使用 rsi。out 是指针，虽然它指向 double，也要走整数/指针参数通道；它是第二个此类参数，不是第四个通用寄存器参数。

一种只展示主体的指令序列是：

```
cvtss2sd %xmm1, %xmm1     # y: float -> double
addsd    %xmm1, %xmm0     # x + y
cvtsi2sdl %edi, %xmm2     # 32位有符号 n -> double
addsd    %xmm2, %xmm0     # 再加 n
movsd    %xmm0, (%rsi)    # 写入 *out，题设 out 有效
ret                       # xmm0 同时承载返回值
```

不要用整数的 `addq` 来做 IEEE 浮点加法。两种数都存在位模式，但数值编码规则不同。

浮点比较还有 NaN：对 `ucomisd %xmm1,%xmm0`，比较的是 xmm0 与 xmm1。结果如下：

关系	ZF	PF	CF
小于	0	0	1
等于	1	0	0
大于	0	0	0

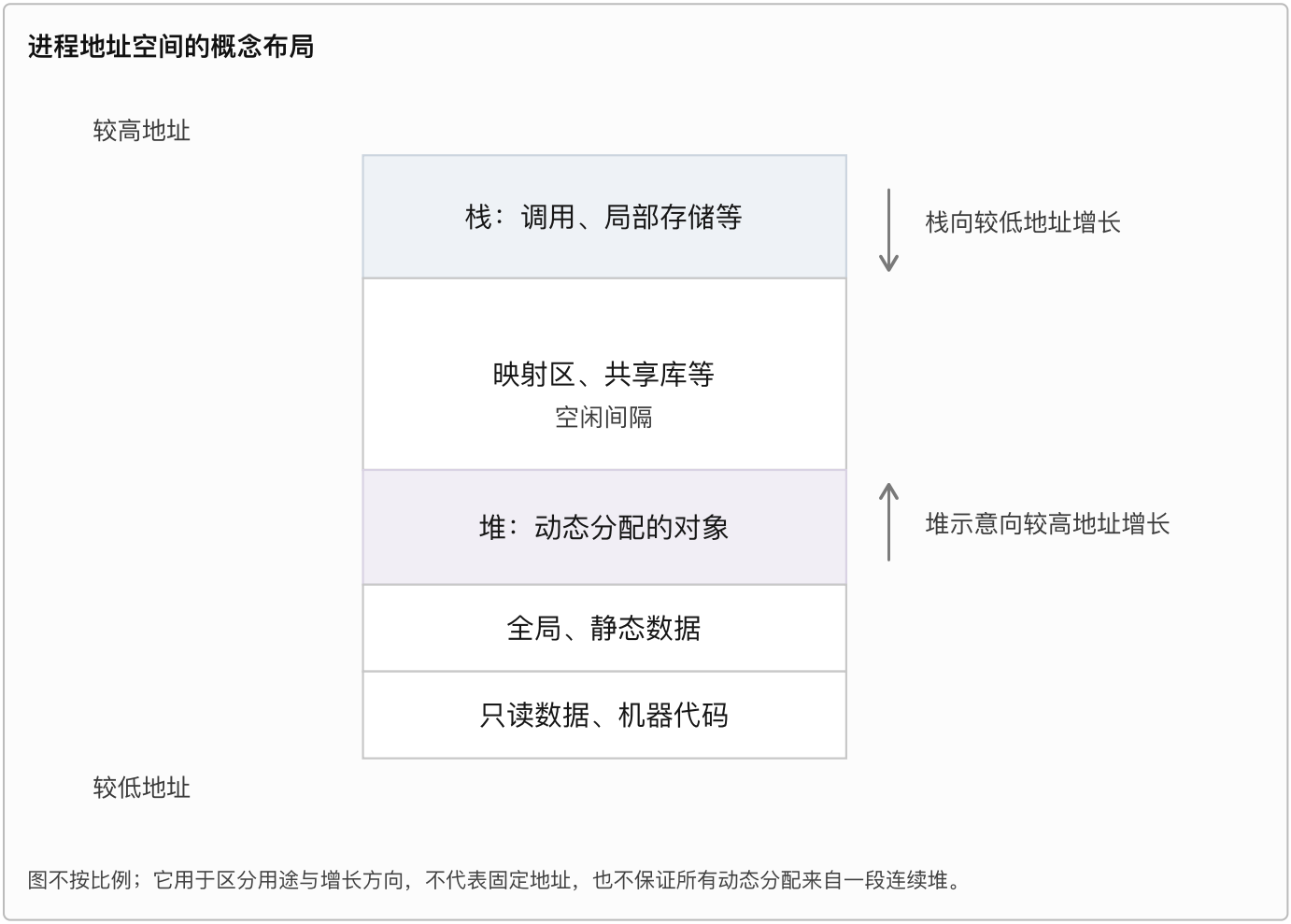
关系	ZF	PF	CF
无序，任一操作数为 NaN	1	1	1

因此只看 ZF=1 不能区分相等和无序。题目若允许 NaN，读代码时还要留意奇偶标志 PF 的分支，例如 `jp`。这里 OF、SF 被清零，不能照搬整数 `jl` 的符号比较套路。

5 程序内存布局与越界

内存布局图描述的是进程的地址空间。具体地址与相对间隔随平台、链接和随机化变化，不要求背某台机器的固定地址。

图4–5 常见布局示意。 这是一张概念图，不按比例；“堆向上、栈向下”是课程常用模型，不代表所有分配都只来自单一连续堆。



一个变量的“作用域”和存储位置是两件事。函数里的局部变量可能在寄存器，也可能在栈；局部指针可以指向堆对象。不要把“函数里出现的所有东西”都画到栈上。

字符串还需要结束的一个零字节

`char buf[8]` 最多容纳七个普通字符和一个 `\0`。`\0` 是一个值为零的字符，不是两个字符反斜杠和零。

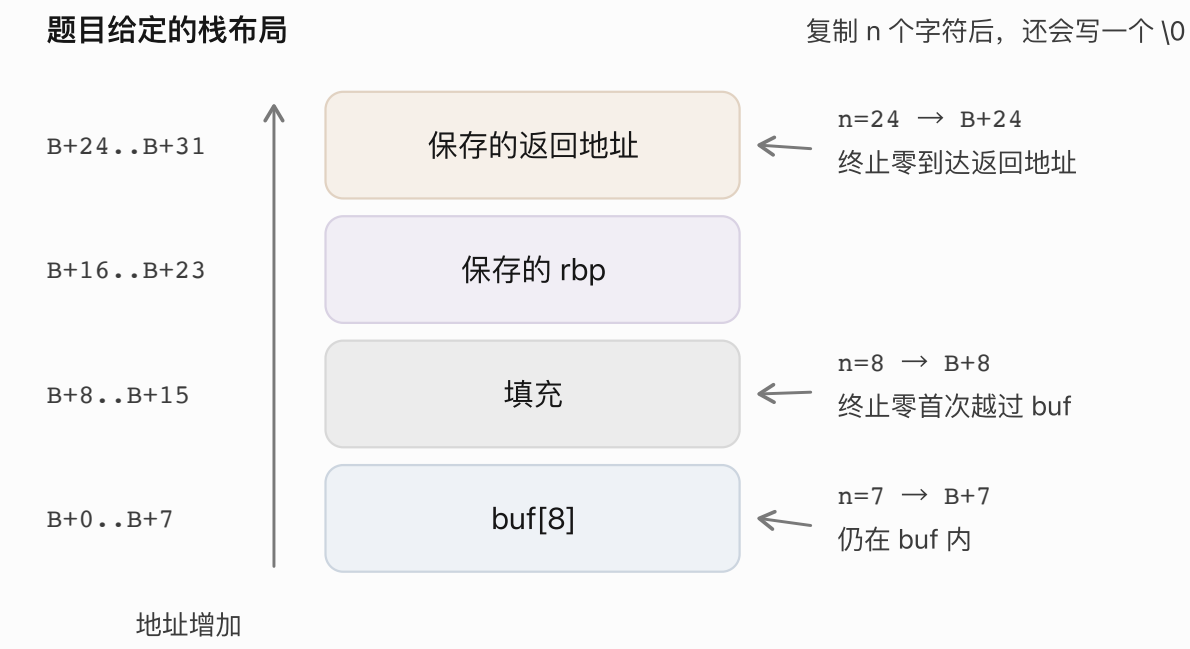
`char a[8]="1234567"` 有终止零；C 中 `char b[8]="12345678"` 的八个位置都被字符占满，没有终止零，不能安全地把 b 当成普通以零结束的字符串传给任意字符串函数。

例 D4–E5 根据给定栈图数越界长度

> Q029：怎样读栈图、数结束零，为什么7、8、24个字符的结果不同?

助手补充例。题目明确给出下列布局；不要把它当成所有编译器的固定布局。

从 buf 写起：终止零也占一个字节



从最低地址 B 向上写；最后一个零字节位于 B+n，所以 n=8 时已经第一次越界。

假设一次不检查边界的字符串复制从 B 开始，依次写入 n 个非零字符，最后再写 `\0`。它总共写 n+1 字节，最后一字节写到 B+n。

- n=7：写 B 到 B+7，恰好容纳，未越界。
- n=8：字符填满缓冲区，终止零写到 B+8，第一次越界已经发生。
- n=24：普通字符写到 B+23，终止零写到返回地址的第一个字节 B+24。

“没有立刻崩溃”不说明写入合法。返回地址被改变后，函数的 ret 可能从栈中取出错误的下一条指令地址。C 层面的越界属于未定义行为；试题给栈布局时，是在分析指定机器模型上的可能效果，不是承诺任何 C 程序都表现一样。

防护方法分别改变了什么

> Q030：长度检查、ASLR、NX和canary分别在防什么?

方法	主要作用	不能据此推出
正确检查长度、使用合适的有界接口	避免越界写入本身	只要函数名有 n 就一定安全
ASLR，地址空间布局随机化	增加预测相关地址的困难	所有越界都会消失
NX/不可执行数据页	阻止直接执行被标为不可执行页中的字节	已有可执行代码不能被重新利用

方法	主要作用	不能据此推出
栈 canary	返回前检查某个保护值是否被破坏，发现特定栈写坏	所有内存错误都会被发现，或已经写坏的数据会自动恢复

`fgets(buf,sizeof buf,stdin)` 在空间参数正确时最多读取 size−1 个字符并终止字符串；还需处理失败、残留输入等使用问题。`strncpy` 并不保证在所有输入长度下都追加 `\0`。canary 通常通过比较“是否相等”发现改变，不是比较数值是否更大。

NX 需要处理器的执行权限检查能力与系统配置配合；现代题设已具备该能力时，启用它不等于必须买一颗新 CPU。遇到旧卷的“是否需要新硬件”，要分清“需要硬件支持”和“当前机器需要更换硬件”。

6 ROP 只需先看懂 ret 链

普通 call 把返回地址压栈，ret 把栈顶八字节取作下一执行地址。ROP 利用同一个机制把已有可执行代码中的短片段连接起来；片段通常以 ret 结束，称为 gadget。它不要求把栈上的数据当指令执行，所以仅有不可执行栈并不足以阻止这种控制流。

本讲只做题目给定片段的状态追踪。代码位置是否可预测取决于具体二进制与随机化配置，不把“所有代码地址固定”当成今天的普遍结论。

例 D4–E6 每次到底弹出什么

助手补充例，假定两个片段如下，STOP 是停止观察的位置：

```
G1: popq %rax
    ret
G2: addq %rax, %rbx
    ret
```

初始 rbx=5，rsp=0x1000，即将执行一个 ret。题给栈内容：

图4-6 返回链与数据槽。 所有槽都是八字节，注意常数 7 是数据，不是跳转地址。

ROP 返回链：初始栈与完整执行状态

0x1018	STOP	G2 末尾 ret 读取
0x1010	G2	G1 末尾 ret 读取
0x1008	7	G1 的 pop 读取，这是数据
0x1000	G1 ← rsp	初始 ret 读取

步骤	rsp	rax	rbx	下一位置
起点：即将执行初始 ret	`0x1000`	未知	5	初始 ret
初始 ret：转到 G1	`0x1008`	未知	5	G1 的 popq %rax
G1 的 pop：rax = 7	`0x1010`	7	5	G1 的 ret
G1 的 ret：转到 G2	`0x1018`	7	5	G2 的 addq %rax,%rbx
G2 的 add：rbx = 12	`0x1018`	7	12	G2 的 ret
G2 的 ret：到达 STOP	`0x1020`	7	12	STOP

1. 本步尚未执行。rsp 指向 G1 所在槽；7 是下一个槽里的数据，不是代码地址。
2. 初始 ret 读出 0x1000 处的 G1，控制转到 G1；rsp 加 8 到 0x1008。
3. pop 从 0x1008 读出数据 7 放入 rax；rsp 再加 8 到 0x1010。控制仍在 G1。
4. ret 从 0x1010 读出 G2，控制转到 G2；rsp 加 8 到 0x1018，rax 仍为 7。
5. add 只把 rbx 改为 5+7=12，不从栈弹出数据，所以 rsp 仍为 0x1018。
6. 最后的 ret 读出 0x1018 处的 STOP；rsp 加 8 到 0x1020，观察在 STOP 结束。

执行步骤	结果	执行后 rsp
初始 ret	下一条指令是 G1	0x1008
G1 的 pop	rax=7	0x1010
G1 的 ret	下一条指令是 G2	0x1018
G2 的 add	rbx=5+7=12	0x1018
G2 的 ret	转到 STOP	0x1020

做题时先按执行顺序列出 pop 与 ret，每次各消耗一个八字节槽。不要把所有栈槽都当作返回地址。

7 独立练习

以下均为助手补充题，数值与示范不同。必做 D4-P1—P8 共40分，32分为过关参考；其中短题每道3分，两道中题各11分。先独立作答，再查看第4讲练习答案。

D4-P1 联合与字节序 3分

`union { unsigned char b[4]; unsigned int w; } u;`。依次写入 b 为 0x01、0x20、0x03、0x40。按本讲的 x86-64 小端前提，求 w；若明确改为大端，求 w；求联合大小。每问1分。

D4-P2 数组与 sizeof 3分

有 `int a[3][4]; int (*p)[4]=a;`。求 `sizeof(a)`、`sizeof(p)`、p+1 比 p 的地址增加的字节数。每问1分。

D4-P3 浮点参数 3分

函数 `double shift(double *p,long k,float x)` 中，p、k、x 在入口时各用哪个寄存器传递？每问1分。额外自检：返回的 double 用哪个寄存器，不另计分。

D4-P4 字符串边界 3分

`char buf[12]` 起于 B，题设保存的返回地址从 B+24 开始。不检查长度地写入 n 个非零字符，再追加零。求：不越界的最大 n；第一个越界的 n；使终止零恰好写到返回地址首字节的 n。每问1分。只做纸面分析，不运行越界代码。

D4-P5 防护识别 3分

分别指出：哪种机制增加地址预测难度；哪种机制检查栈中的保护值是否改变；为何不可执行栈仍不能单独阻止利用已有代码的返回链。每问1分。

D4-P6 返回链追踪 3分

G1 是 `popq %rax; ret`；G2 是 `leaq (%rax,%rax,2),%rax; ret`。初始 rsp=0x800，即将执行 ret；地址 0x800、0x808、0x810、0x818 中依次是 G1、9、G2、DONE。求：进入 G1 时 rsp；执行 G2 的 lea 后 rax；转到 DONE 后 rsp。每问1分。

D4-P7 二维数组中题 11分

`int a[3][5]` 首地址为 0x2000，所有下标合法。

1. 写出 `&a[i][j]` 的一般地址公式。(2分)
2. 求 `&a[2][3]`。(2分)
3. 求 `a+1` 的地址和 `sizeof(a)`。(2分)
4. 假设符号 TAB 指向该数组，i 在 rdi，j 在 rsi，补齐主体片段：(3分，每行1分)

```
leaq (____,____,____), %rax  # 先求5*i
addq ____, %rax
movl TAB(,____,____), %eax
```

5. 若换成 `int *rows[3]`，为什么不能继续无条件使用相同的二维数组地址公式？ (2分)

D4-P8 结构体中题 11分

```
struct Record {
    char c;
    short s;
    long n;
    int a[3];
};
```

1. 求四个字段的偏移。(2分)
2. 求整体对齐和 sizeof。(2分)
3. 若对象首地址为 0x3000，求 a[2] 的地址。(2分)

4. 在 `struct Record r[2]` 中，第二个对象比第一个对象晚多少字节？为何不能只加各字段原始大小之和？（2分）
5. 另有 `struct T { char c; int a[N]; long z; };`，N 为正整数。若 z 的偏移为16，求所有可能的 N，并说明为何不能仅取最大的一个。（3分）

D4–X1 达标后选做

`double update(long *p,double x,float y)` 在指定顺序下计算 `r=(double)*p+x+(double)y` 并返回。写出三个参数和返回值的寄存器；写出读入 *p、转换为 double、两次加法所需的主要指令。假设指针有效，遵守 C 表达式的加法顺序。不计入必做分数。

8 收尾与来源

独立检查五件事：能否从类型确定步长；能否逐字段算对齐；能否从字节序恢复整数；能否区分浮点值和指向浮点的指针；能否按栈槽追踪 ret。错了哪一项，就回到对应的图和例题，再用新数值检验。

本讲的解释、图示和未特别标注的例题属于助手补充；概念依据如下，页码都是 PDF 文件页序，不是教材印刷页：

- 课程第7讲（来源资料未随仓库分发）：第3—23页数组与指针，第25—35页结构体和对齐，第37—42页浮点机器代码，第45—51页声明和 sizeof。
- 课程第8讲（来源资料未随仓库分发）：第3—11页联合，第13—16页内存布局，第18—33页越界，第40—47页防护，第48—53页ROP。
- 教材（来源资料未随仓库分发）：3.8，3.9，3.10.1，3.10.3—3.10.4，3.11相关部分。这里使用小节定位，不混写文件页序和印刷页码。
- 第7讲研讨题（来源资料未随仓库分发）：第1—2页普通研讨与浮点机器编程部分，不采用拓展研讨。

第5讲 完整测验与考前整理

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

1 开始前只做这几件事
2 做卷时的固定方法
3 本次训练的评分方式
4 失分后如何补
5 纠错用的新题库
6 最终速查单
7 2025第一次阶段测验：24题完整解析

这是复习计划的第5天。今天的主要任务是检验能否独立作答，然后修复最影响得分的问题，不再增加新的知识专题。

按你的要求，本讲已加入2025第一次阶段测验全部24题的完整解析：直接查看逐题解析。网页中每题可单独展开；解析依据本地带答案版，并标注题面笔误与答案口径差异。

1 开始前只做这几件事

使用2025 第一次阶段测验无答案版（来源资料未随仓库分发），共10个PDF文件页、24题。若选择独立模拟，先完成作答再展开答案；也可以按需逐题学习解析。若此前已经看过某道题的答案，在答卷上标“已见”，最后与未见题分开统计。

本地原卷没有注明考试时长，训练暂设120分钟；若老师已经公布实际时长，优先采用官方时长。限时内不查讲义、不运行代码、不请求提示；看不懂题面可以先标记。时间结束画一条分界线，之后补做的内容不能倒算为限时内得分。

开做前统一下面几处题面口径。这些是为了减少材料歧义，不是新增考试要求：

- 第3题“首字节”在本次训练中指**最低内存地址处的字节**。
- 第17题 `bit_not1(long a)` 内的 `~p` 改为 `~a`，`long y=~x` 后补分号。按课堂“取地址的变量需要可寻址存储、其他变量尽量用寄存器”的示意模型作答，不推断某一优化级别的真实编译输出。
- 第12题的子编号有跳跃，按原卷出现的空作答，不需要寻找不存在的第3问。
- 第18题内存操作数里印刷缺少的寄存器前缀 `%` 可自行补上。汇编填空按整行检查，不能把一行中的多个空重复计成多行。
- 第20题按课上给出的连续二维数组机器布局计算；不把此例当作可移植C跨行指针算术的证明。
- 第24题涉及旧材料的硬件前提。“引入执行权限检查是否需要硬件能力”与“已经支持该能力的机器是否要更换CPU”是两种口径。本次训练接受明确区分前提且解释正确的答案。

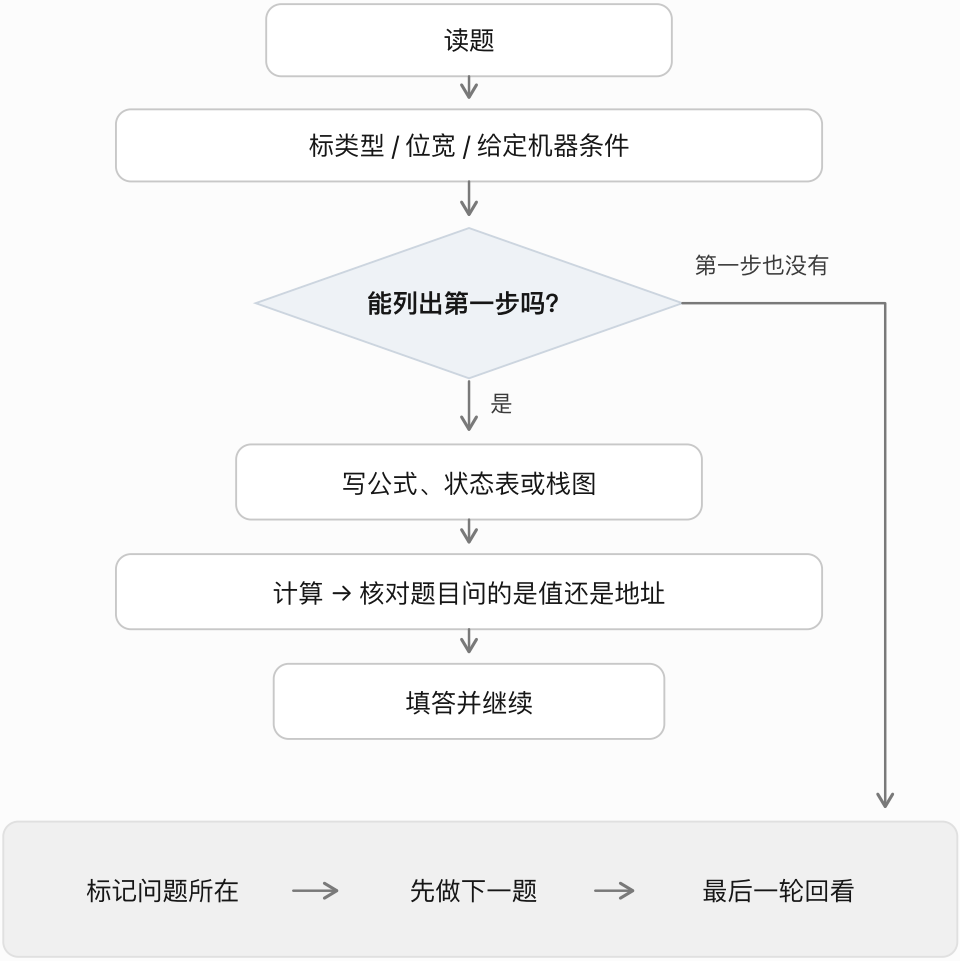
这些说明没有改写原PDF；均为本套复习训练的统一说明。

2 做卷时的固定方法

先快速浏览题型，直接拿下能确定的题；一题开始无进展时先写已知关系，再标记跳过。返回时优先处理“只差一步”的题。最后检查单位、位宽、signed/unsigned、寄存器宽度、十六进制进位、数组下标和答案填写位置。

图5–1 做题时的判断流程。 箭头表示下一步动作，遇到卡点时不必在原地反复猜。

考场上的下一步如何决定



能写第一步就落笔推理；完全无从下手时先标记，把时间留给能推进的题，最后统一回看。

不同题型的第一步尽量固定：

题型	第一笔写什么
C整数表达式	每个操作数的类型，提升或转换后再比较
浮点	s、exp、frac，以及属于哪一类编码
汇编片段	输入寄存器、输出寄存器、访问宽度
控制流	cmp实际计算谁减谁，分支是有符号还是无符号
栈	当前rsp、每次push/call/pop/ret的变化
数组	首地址 + 元素大小 × 线性下标
结构体	每字段大小、对齐要求、放置前后的偏移
内存安全	起始地址、写入总字节数、目标位置偏移

3 本次训练的评分方式

原卷第5、6讲的逐空/逐行规则与分组总分不完全一致。为避免为了凑100分随意删题，这里先按“作答单位”记录，再按原卷各讲总分换算。这是**训练分**，不是重新宣称官方评分规则。

一个单位原则上得0或1；说明题只写对结论、缺少关键原因可得0.5。具体单位与答案在核对文件中列明。记第k讲得分单位数为c_k：

原课程讲次	题号	单位总数	训练分换算
2	1—3	6	$12\times c_2/6$
3	4—6	10	$10\times c_3/10$
4	7—9	6	$6\times c_4/6$
5	10—14	15	$28\times c_5/15$
6	15—17	17	$16\times c_6/17$
7	18—22	10	$20\times c_7/10$
8	23—24	4	$8\times c_8/4$

各组换算后相加，最后保留一位小数，满分100。第18、19题各两行汇编分别按两单位计；第23题完整输出是一单位；第24题每个待填行是一单位，已给出的示范行不计。

作答后再打开保留卷核对与解析。遇到题面歧义而做出合理解释时，写明依据再评定，不机械照抄旧答案。

4 失分后如何补

先列一张自己的记录表，保留空白直到实际做完。不把“讲过”自动填成“已掌握”。

题号	限时内是否完成	首次错误	归类	应回看哪一讲	新题复测结果
实际作答后填写					

错误分四类：概念（不知道规则）、步骤（知道但推算错）、审题（看错类型或前提）、时间（会做但没有完成）。先修本次失分最多、能够短期改正的两类问题。每类先用一个小例子讲清，再从下方选择对应的两道新题独立验证。

重复原题可以确认有没有理解解析，但不能代替新题检验。训练分85%是检查目标，90%以上是冲刺目标；低于目标时，按具体失分补，不把总分高低当成唯一反馈。

5 纠错用的新题库

以下全部为助手补充，供测验后选取；不是额外要求把14题全部刷完。每组两题，选择最薄弱的两组，共四题。只看题面，做完后查纠错题答案。

A 整数与类型

D5–A1

一个8位位模式为0x92。分别按无符号和补码解释；将补码值符号扩展到16位后，位模式是什么？

D5–A2

在本课程32位int模型下，`int x=-3; unsigned u=2;`，判断 `x<u`，并写出比较前x对应的无符号数值。不要只回答真或假。

B 浮点

D5–B1

一种8位格式由1位符号、3位阶码、4位小数组成，bias=3，编码分类按IEEE式规则。位串`0 010 1000`表示什么数？`0 000 0010`又表示什么数？

D5–B2

按最近偶数舍入，把二进制1.101100保留三位小数。说明为何需要或不需要进位。

C 寻址与寄存器

D5–C1

rax=0x200，rcx=5，求`0x18(%rax,%rcx,8)`的有效地址。若它出现在lea中，是否会读取该地址的内存？

D5–C2

rax原值0xFFFFFFFF12345678，执行`movw $0xABCD,%ax`后是什么？另一次从原值重新开始，执行`movl $0xABCD,%eax`后是什么？

D 控制流

D5–D1

执行`cmpq %rsi,%rdi; setb %al`，对应的无符号关系是什么？若题目改为有符号的“小于”，应改用哪个set指令？

D5–D2

下列按无符号64位计算的片段返回什么函数结果？输入n在rdi。

```
movq %rdi, %rcx
xorl %eax, %eax
.Lagain:
testq %rcx, %rcx
je .Ldone
addq %rcx, %rax
subq $1, %rcx
jmp .Lagain
.Ldone:
ret
```

请写出对应循环；计算n=4和n=0的结果。

E 调用栈

D5–E1

调用者准备执行call时rsp=0x700，下一条指令地址为0x401025。被调用者按顺序执行`pushq %rbx; subq $16,%rsp`。此时rsp是什么？保存的返回地址相对于当前rsp偏移多少？

D5–E2

函数先用rbx保存一个稍后还要使用的值，然后调用另一个遵循ABI的函数。能否依赖调用后rbx仍保持原值？本函数自己改写rbx时，对它的调用者又有什么责任？

F 数据布局

D5–F1

`short a[4][6]`首地址0x5000，求`&a[3][2]`。写出乘的是列数还是行数。

D5–F2

`struct T {char c; double d; int i;};`按本套约定，求成员偏移、整体对齐和总大小。

G 联合与内存安全

D5–G1

四个字节按地址升序为AA、BB、CC、DD，按小端unsigned int读取为何值？这是否等于把每个字节的数值相加？

D5–G2

已把栈设置为不可执行，为什么仍应检查写入长度？请分别从数据正确性和返回到已有代码这两方面说明。

6 最终速查单

这里是最小通用版。考前再加入自己确实错过的规则，不扩成另一份长教材。

知识	最后核对
整数	先类型后运算；同一位模式可有不同解释；C有符号溢出不等于语言保证回绕
补码	w位范围为−2^(w−1)到2^(w−1)−1；负值符号扩展补1
位移	先检查位宽、类型、移位量；负数算术右移的舍入方向与C除法可能不同
字节序	小端低地址放低位字节；画地址方向，不靠背字符串顺序
浮点	先分类再算E与M；exp不是E；非规格化没有隐藏1；中点取保留位末位为偶数的一侧
汇编	AT&T源在前、目的在后；lea不读被计算地址的内存；写eax清rax高32位
条件	cmp S,D看D−S；signed与unsigned分支不混用；cmov不自动让危险求值消失
循环	初始化、条件、更新、循环体四项；注意先测还是后测
过程	call压下一条指令地址；ret弹地址；整数返回值常在rax；返回地址不是返回值
数组结构	元素大小×线性下标；每字段先对齐，结构整体最后再对齐
浮点代码	指针仍走通用寄存器；浮点参数走XMM；ss是单精度，sd是双精度
越界	n个字符常需n+1字节；“没崩”不等于合法；各防护解决的问题不同

最后能够口头讲清：一道转换、一道舍入、一个条件分支、一张调用栈图、一个数组地址和一个结构体布局。本轮学习结束后，将真实的未解决问题写入对应的“第几讲后补充讲义”，不要把推测的弱点预填成记录。

7 2025第一次阶段测验：24题完整解析

本节包含全卷答案，按2026-10-11的实际学习请求加入。可以按题号展开，留在第5讲内阅读；若正在做独立模拟，先完成相应题目再核对。

题目来源：无答案版（来源资料未随仓库分发）；参考答案：带答案版（来源资料未随仓库分发）。每题的推导、代码注释与易错点由助手补充，不是原试卷发布方的新勘误。题号按1—24排列，“原课程第几讲”与本套复习讲次不是同一编号。

下方可逐题展开；需要从头连续阅读时，也可以打开全卷连续解析。打印版的完整解析仍放在独立答案册。

阅读前注意三处口径：第3题参考写0xFF，但若“首字节”指小端最低地址字节，应为0xF4；第23题参考数值正确，实际%x输出小写字母；第24题需要区分硬件能力与是否必须更换现有CPU。第10、17、18题的代码笔误及其他语言前提也在对应题中说明。